

tuto 1



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3843 - SPECIAL TOPIC IN DATA ENGINEERING

SECTION 2

MS AZURE TUTORIAL REPORT

LECTURER: DR. ARYATI BINTI BAKRI

NAME	MATRIC NUMBER
NURUL IKA SYAFINY BINTI AZHAR	A23CS0164
DAMIYA AINA BINTI BASIR ABD SHAMMAD	A23CS0220

1.0 Project Overview

Problem Background

In the modern data landscape, organizations struggle to unify fragmented data sources into actionable insights. Traditional on-premise data systems often lack the scalability and agility required to process large-scale datasets efficiently. This project addresses the challenge of building a robust, cloud-native infrastructure that can automate the journey of data, from its raw, unstructured state to a polished, visual format without the overhead of maintaining physical hardware.

Project Objectives

The primary goal of this project is to implement a fully automated data pipeline using Microsoft Azure. The specific objectives include:

- **Data Ingestion:** Establishing a secure connection to ingest raw data from external sources into a centralized cloud storage environment.
- **Pipeline Automation:** Leveraging Azure Data Factory to orchestrate seamless data movement and transformation.
- **Data Transformation:** Processing and cleaning data to ensure it is structured and ready for analytical consumption.
- **Actionable Analytics:** Developing an MS Power BI dashboard to provide meaningful visualizations and data storytelling for business stakeholders.

Business and Industry Relevance

This solution mirrors real-world industry requirements where data engineering is the backbone of business intelligence. By utilizing a "Bronze, Silver, and Gold" data architecture in the tutorial, businesses can ensure data integrity and high-speed reporting. Implementing this on Microsoft Azure demonstrates a cost-effective, scalable approach to data management that allows companies to make data-driven decisions in real-time, providing a significant competitive advantage.

Project Scope

The scope of this project encompasses the end-to-end development of a data pipeline. It begins with raw data storage in Azure Data Lake, coding for silver and gold data processing in Azure Databricks, moves through building the data pipeline architecture via Azure Data Factory, and concludes with the deployment of an interactive Power BI reporting layer. The project is limited to the functionalities provided within the MS Azure environment.

2.0 Description on The Solution

An end-to-end cloud-based data engineering pipeline built with Microsoft Azure services is the solution created in this tutorial demonstrating. In order to ensure effective data processing and analysis, the system is built to automate every stage of the data lifecycle from data ingestion to final visualization.

The implementation follows the Medallion Architecture, consisting of Bronze, Silver and Gold layers. Raw data files were absorbed at the Bronze layer and kept in their original format in Azure Data Lake Storage. This layer acts as the first landing zone, guaranteeing the unaltered preservation of all source data.

Next, the Silver layer is responsible for data cleaning, transformation and integration. Azure Databricks is used to process raw data in order to apply business logic, handle missing information and eliminate inconsistencies. This step guarantees that the data is consistent, organized and prepared for additional analysis.

The final, analytics-ready dataset is represented by the Gold Layer. Azure SQL Database stores the processed data in an organized format, usually with reporting and query-optimized schemas. This makes it possible to obtain high-quality data quickly and effectively. Azure Data Factory orchestrates the pipeline in order to automate the workflow. It controls the flow of data between layers and guarantees that every operation operates on time and dependably.

Lastly, the visualization layer is connected with Microsoft BI. It allows users to make data-driven decisions by connecting to the Gold layer data and displaying insights through interactive dashboards. All things considered, this solution shows how to use cloud technologies for current data engineering in a scalable, automated and effective manner.

3.0 Technology Used

This project leverages the Microsoft Azure ecosystem to build a scalable, cloud-native data engineering solution. Each service was selected for its ability to handle specific stages of the data lifecycle, ensuring high availability and seamless integration.

- Azure Data Lake Storage (ADLS) Gen2 Storage Account: Serves as the primary storage layer for the project. It is used to implement a tiered storage architecture where we store Bronze, Silver, and Gold folders as containers, allowing for the separation of raw, cleansed, and curated datasets.
- Azure Data Factory (ADF): Acts as the central orchestration engine. It is used to create and manage data pipelines that automate the movement of data from the source to the storage layers through "Copy Data" activities and "Triggers". The medallion architecture is applied using this application.
- Azure SQL Database: Provides a relational storage environment for the final, curated data. This service hosts the "Gold" layer data, making it easily accessible for high-performance querying and reporting.
- Azure Key Vault: Used to enhance security by securely storing and managing secrets, such as database connection strings and access keys, ensuring that sensitive information is not hard-coded within the ADF pipelines. It is used to store all the sensitive keys and passwords that are used in this tutorial like the SQL Server Management Studio password, username and the databricks key.
- Azure Databricks Service: Employed for advanced data transformation and cleaning. Using Python language notebooks in Databrick, data is processed at scale to handle transformations from bronze to silver and silver to gold.
- Azure Synapse: Used as the primary orchestration engine to automate the end-to-end workflow. We implemented dynamic pipelines using Get Metadata and ForEach activities to programmatically iterate through data layers, ensuring that the creation of SQL views remains automated and scalable as new datasets are introduced. It uses a Serverless SQL pool to create views over the Gold Delta files, allowing Power BI to query the data using standard SQL without the need for a dedicated database.
- Microsoft Power BI: Used for the final visualization layer. It connects directly to the Azure SQL Database to transform curated data into interactive dashboards for business storytelling.
- Azure Active Directory (Entra ID): Utilized to provide a secure identity framework. By using Managed Identities, eliminate the need for hardcoded credentials in the pipeline, and RBAC roles were assigned to ensure that only authorized services could access the sensitive data stored in the Bronze, Silver, and Gold layers.

4.0 Data Pipeline Architecture

The architecture follows a Medallion Architecture implemented on the Microsoft Azure Cloud. This design ensures data integrity as it moves from raw on-premises sources to a refined state ready for business intelligence. The architecture diagram applied in this tutorial is shown in Figure 1.

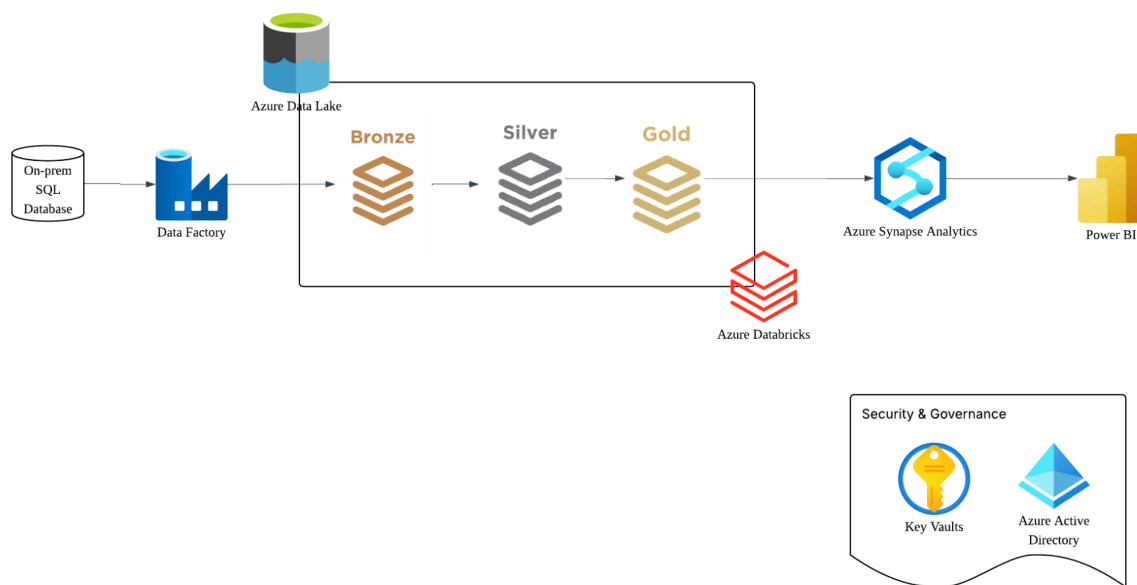


Figure 1: Azure Tutorial Data Pipeline Architecture

1. Data Ingestion (On-Prem to Bronze)

- Source: The pipeline begins with an On-premises SQL Server (SSMS).
- Ingestion Tool: Azure Data Factory (ADF) acts as the orchestration engine. It utilizes a Self-Hosted Integration Runtime (SHIR) to securely bridge the gap between the local private network and the Azure cloud.
- Destination (Bronze Layer): Data is landed in its rawest form in Azure Data Lake Storage Gen2 (ADLS Gen2). This "Bronze" layer acts as a historical record, preserving the original state of the source data in Parquet format.

2. Data Transformation (Silver and Gold)

- Processing Engine: Azure Databricks is utilized for its high-performance Spark clusters.
- Silver Layer (Cleansing): Databricks notebooks read from the Bronze layer to perform data cleansing. This includes handling null values, correcting data types, and implementing the snake_case naming convention for standardizing column headers.

- Gold Layer (Curated): The cleaned data is then transformed into the Gold layer. Here, the data is aggregated and structured into business-level entities (Fact and Dimension tables), optimized for reporting performance and stored in Delta format for ACID compliance.

3. Data Serving and Visualization

- Serving Layer: Azure Synapse Analytics provides a Serverless SQL Pool that acts as a relational "view" over the Gold Delta files. This eliminates the need to move data into a heavy database, reducing costs while providing a standard SQL endpoint.
- Visualization: Power BI connects to the Synapse endpoint. This allows for the creation of interactive dashboards where stakeholders can derive insights from the processed data.

4. Security and Governance

- Azure Key Vault: To adhere to security best practices, sensitive credentials like SQL passwords and storage keys are never hardcoded. They are stored as secrets in Key Vault and retrieved dynamically by ADF.
- Azure Active Directory: Provides the security backbone through Identity and Access Management (IAM) and Managed Identities, ensuring that only authorized services can communicate across the pipeline.

5.0 MS Power BI Dashboard

Using data from an Azure-based data pipeline, the Power BI dashboard created for this project provides a comprehensive summary of sales performance. With slicers and filters, the dashboard's essential visual components such as total sales, product category breakdown, and product-level analysis allow for interactive data exploration.

Total Sales, a crucial KPI in the dashboard, is computed dynamically using a DAX measure based on order quantity and unit pricing from the SalesOrderDetail table. This guarantees that, following a data refresh, any database adjustments are instantly reflected in the visualization layer.

The total sales value was roughly 700.69K prior to the insertion of a new sales record. The Total Sales rose to about 714K when a new transaction was added to the SQL database and the Power BI dataset was refreshed. This shows that the dashboard can reflect changes made in real time via the data pipeline and is correctly connected to the underlying data source.

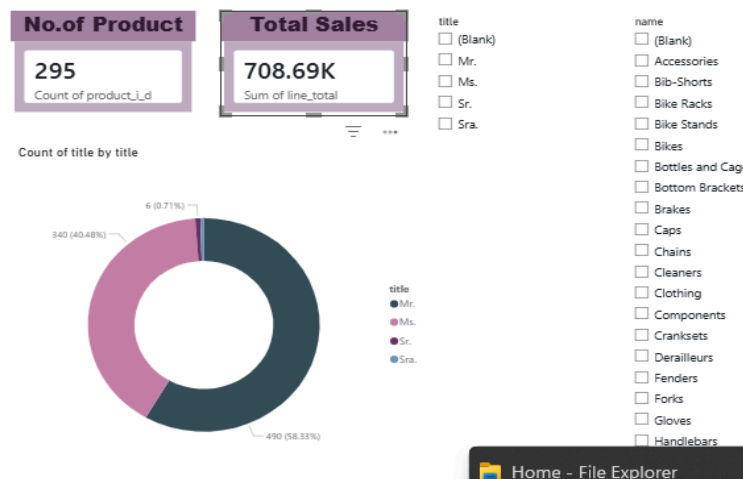


Figure 2: Dashboard with Original Data

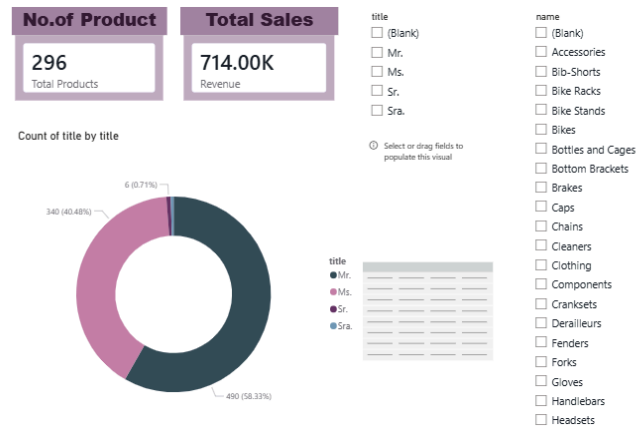


Figure 3: Dashboard with Updated Data

Additionally, the dashboard has interactive slicers for product title and category that let users dynamically filter sales data. Additionally, visual elements like donut charts and cards offer both comprehensive and concise views on product distribution and sales contribution.

Activities Performed in Power BI Dashboard Development:

Several crucial tasks were completed throughout the dashboard's development:

1. Imported data from SQL Server (the AdventureWorksLT2025 database) that had been cleansed and organized.
2. Established connections between several tables, including Customer, SalesOrderDetail, Product, and ProductCategory.
3. Developed a paradigm that resembles a star schema to guarantee effective reporting and filtering.
4. To compute important metrics like Total Sales, DAX measures were developed.
5. Created interactive graphics, such as:
 - KPI cards (Product Count/Total Item Sold, Total Sales)
 - Donut charts (distribution of products)
 - Slicers (product and category screening)
6. By adding a new entry to the SalesOrderDetail table and verifying the modifications in Power BI following a refresh, data updates were tested.

6.0 Reflection

Damiya Aina binti Basir Abd Shammad

This project gave me hands-on experience utilizing Azure and Power BI to create a comprehensive data engineering and analytics pipeline.

First, I gained knowledge about how Azure Data Factory pipelines operate in practical situations, particularly with regard to the transfer and transformation of data between various tiers. I also realized how crucial proper configuration is, since little problems with permissions or data mapping can have an impact on the pipeline as a whole.

Second, I gained a deeper comprehension of Power BI, especially with regard to data modeling, relationships, and DAX computations. At first, I had issues where wrong fields and relationships caused visualizations to update incorrectly. Nonetheless, this enhanced my attention to detail and troubleshooting abilities.

Last but not least, this project helped me better grasp how business intelligence tools work with cloud-based platforms. I can see how important insights that aid in decision-making can be derived from unprocessed data. Overall, this project strengthened both my technical skills and problem-solving ability in data engineering and analytics.

Nurul Ika Syafiny Binti Azhar

Prior to this tutorial, I viewed cloud services as individual tools. Through this project, I have learned how to orchestrate a complex ecosystem. I gained hands-on experience in configuring a Self-Hosted Integration Runtime, which taught me the realities of connecting local, on-premises data to the cloud. I now understand how Azure Data Factory acts as the "heart" of the system, timing the movement of data so that Databricks and Synapse always have the most current information.

The most valuable part of this project was overcoming technical hurdles. For example, I encountered errors where "Test Connection" succeeded, but the actual pipeline run failed. This taught me that credential validation is only one step; permissions (IAM) and firewall rules are equally critical for data movement. Encountering "Unique Key Violations" in SSMS forced me to dive deeper into database constraints and the importance of using variables like @x for handling complex data types like varbinary.

Implementing the Bronze, Silver, and Gold layers changed my perspective on data quality. I learned that data is rarely "ready" when it arrives. By using Databricks to transform raw Parquet into cleaned Delta files, I saw firsthand how standardized naming conventions, snake_case and schema enforcement make the final data much easier to use in Power BI.

To conclude this reflection, this tutorial has taught me so much about the service offered in Azure and how I can apply it in future projects like the PPG case study project. Despite the errors I encountered along the way, it is worth the trouble for the knowledge I learned as a future data engineer and the experience I got.

tuto 2



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3843 - SPECIAL TOPIC IN DATA ENGINEERING

SECTION 2

APACHE SPARK TUTORIAL REPORT

LECTURER: DR. ARYATI BINTI BAKRI

NAME	MATRIC NUMBER
NURUL IKA SYAFINY BINTI AZHAR	A23CS0164
DAMIYA AINA BINTI BASIR ABD SHAMMAD	A23CS0220

1.0 Business Understanding

The business background, goals, stakeholders, and anticipated value of the ETL pipeline project are all explained in this section. Understanding the difficulties posed by unprocessed educational census data and defining the analytical objectives necessary for infrastructure analysis and decision-making are the main objectives.

1.1 Problem background

Large-scale educational records gathered from Brazilian schools are included in the `Microdados_Censo_Escolar_2021` dataset. The dataset contains statistics on the availability of digital resources, administrative classification, and school infrastructure. However, because of irregular formatting, repetitive records, missing values, and mixed data types, the raw dataset is extremely detailed and challenging to directly analyze. Analytical querying and reporting procedures become less effective as a result of these problems. In order to put the unprocessed census data into a clear, organized format that can be analyzed and visualized, data pretreatment and transformation are necessary.

1.2 Objectives

This project's goal is to provide a straightforward ETL pipeline that uses Apache Spark to convert unstructured census data into an analytical format.

The pipeline seeks to:

- Extract pertinent school-level information from unprocessed census files
- Create clean analytical datasets from inconsistent and unstructured properties.
- Load structured data into a relational database for visualization
- Use a star schema model to facilitate multidimensional analysis.

Key analytical goals include:

- Analysing internet access availability across schools and states
- Evaluating distribution of teaching staff across administrative categories
- Monitoring student enrollment distribution and demographic trends
- Supporting educational resource planning through dashboard analytics
- Enabling fast aggregation queries for large-scale educational analysis

1.3 Stakeholders

The system is designed for the following stakeholders:

- Education policymakers → to identify infrastructure gaps
- Government agencies → to support resource allocation decisions

- Researchers → to analyse educational inequality
- Data analysts → to build dashboards and reports in Metabase

1.4 Business Value

This ETL pipeline provides significant business value by:

- Improve decision-making: Structured data enables evidence-based evaluation of school conditions.
- Reducing data complexity: Raw census data is transformed into a simplified analytical structure.
- Enabling fast analytics: The final star schema supports efficient aggregation and dashboard performance.
- Supporting digital education planning: Helps track digital infrastructure readiness across schools.
- Creating scalable data architecture: The pipeline can be extended to future census datasets without redesign.

2.0 Data Understanding

The structure, features, and quality of the Brazilian School Census dataset utilized for the study are described in this chapter. Understanding the dataset's characteristics, spotting problems with data quality, and figuring out how the data advances the ETL pipeline's analytical goals are the main goals of the analysis.

2.1 Data Overview

The Brazilian School Census (MICRODADOS_CENSO_ESCOLAR_2021), which includes comprehensive institutional-level records of schools throughout Brazil, is the dataset utilized in this project.

Every record depicts a school entity with characteristics such as:

- Identifiers of institutions
- Geographical location (state, city)
- Administrative categorization (location type, dependence type)
- Indicators of infrastructure
- Digital resources

2.2 Data Structure Analysis

The dataset is distinguished by:

- broad structure with several properties for each record
- mixed data kinds, including binary, numerical, and categorical indications
- Unprocessed, unnormalized format
- The existence of inconsistent and missing values

The dataset is not appropriate for direct analytical queries without preprocessing because of these features.

2.3 Data Type Identification

The dataset attributes are categorized based on their data types to enable efficient data pretreatment and transformation inside the ETL pipeline. The proper cleaning, encoding, and aggregation techniques for further stages of analysis are determined with the use of this classification.

2.3.1 Categorical Variables

These variables describe school identity and classification:

- NO_MUNICIPIO (city / municipality)
- SG_UF (state / federative unit)
- TP_DEPENDENCIA (school administrative type / dependency type)
- TP_LOCALIZACAO (location type: urban or rural)

2.3.2 Numerical Variables

These variables represent measurable quantities:

- QT_SALAS_EXISTENTES (number of classrooms)
- QT_COMPUTADOR (number of computers)

2.3.3 Binary / Indicator Variables

These variables represent infrastructure availability (typically 0/1 values):

- IN_AGUA_POTAVEL (drinking water availability)
- IN_BANHEIRO (bathroom availability)
- IN_BIBLIOTECA (library availability)
- IN_INTERNET (internet availability)

- IN_REFEITORIO (cafeteria availability / dining facility)

2.4 Relational Data Model

The relational data model is structured into three core tables: schools, facilities, and technology, which are linked through the unique identifier school_id (CO_ENTIDADE) . The central entity table providing institutional and geographic data, including school name, location, and administrative classification, is the schools table. The facilities table is related to the schools table through a one-to-one relationship using school_id, and captures physical infrastructure indicators such as classrooms, internet access, water supply, and sanitation facilities. In a similar vein, the technology table, which is linked to the schools table using the same primary key, contains information about digital infrastructure, particularly the quantity of computers that are available in each school. Data can stay non-redundant and well-organized thanks to this structure, which guarantees that all relational tables are consistently linked via a common key.

This relational design plays a crucial role in supporting the subsequent data modelling phase, as it provides a clean and normalized staging layer before transformation into a multidimensional star schema. By separating institutional attributes, physical infrastructure, and digital resources into distinct but connected tables, the model reduces data duplication and improves data integrity. This structured approach ensures that the dataset can be efficiently transformed into fact and dimension tables in Apache Spark, enabling scalable analytical processing and optimized querying in the final data warehouse.

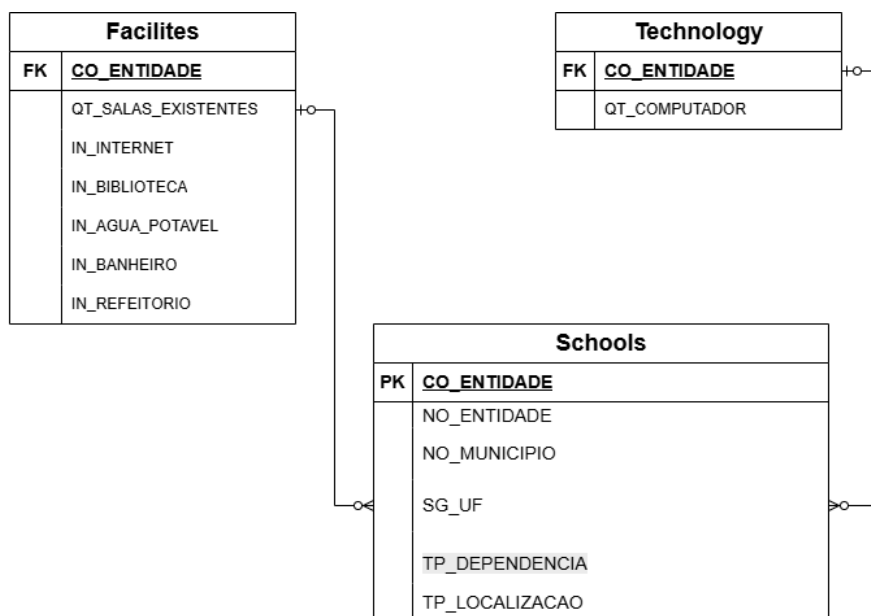


Figure 1: This relational model represents a staging layer prepared for transformation into a star schema (fact and dimension model).

2.4.1 Schools

Represents school identity and classification:

- CO_ENTIDADE (school_id – Primary Key)
- NO_ENTIDADE (school_name)
- NO_MUNICIPIO (city)
- SG_UF (state)
- TP_DEPENDENCIA (school_type)
- TP_LOCALIZACAO (location_type)

Purpose:

Defines the core school entity for relational linking

2.4.2 Facilities Table

Represents physical infrastructure indicators:

- QT_SALAS_EXISTENTES (number of classrooms)
- IN_AGUA_POTAVEL (water access)
- IN_BANHEIRO (bathroom availability)
- IN_BIBLIOTECA (library availability)
- IN_INTERNET (internet access)
- IN_REFEITORIO (cafeteria availability)

Purpose:

Captures school infrastructure conditions

2.4.3 Technology Table

Represents digital resources:

- QT_COMPUTADOR (number of computers)

Purpose:

Measures digital readiness of schools

2.5 Data Quality Issues Identified

The following problems were found and fixed during exploration:

- "Unknown" is used in place of missing categorical data.
- Empty numeric fields converted to 0
- Validation rules are used to clean non-numeric values.

- Eliminated whitespace and uneven formatting
- CO_ENTIDADE was used to eliminate duplicate school records.

3.0 Data Preparation

The preprocessing and modification procedures carried out on the raw dataset prior to its loading into PostgreSQL are described in this chapter. The data was cleaned, standardized, validated, and reorganized into normalized relational tables appropriate for dashboard viewing and analytical processing using Apache Spark.

3.1 Overview

The goal of the data preparation stage is to organize and prepare the unstructured Microdados do Censo Escolar 2021 dataset for analysis. This step guarantees that the dataset is cleaned, standardized, and rearranged into a relational structure appropriate for Metabase's subsequent analytical processing and display.

To guarantee scalability and reproducibility, Apache Spark was used to implement the preparation process.

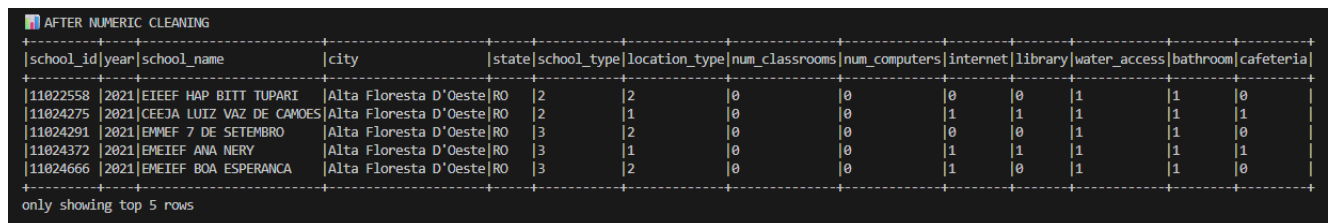
3.2 Data Cleaning and Quality Handling

3.2.1 Raw Original Dataset before Transformation

The original raw dataset put into Apache Spark before preprocessing is depicted in this figure. Geographical information (NO_UF, NO_MUNICIPIO), school information (CO_ENTIDADE, NO_ENTIDADE), infrastructure indicators (IN_BIBLIOTECA, IN_INTERNET, IN_BANHEIRO), digital resource attributes (QT_COMPUTADOR), and enrollment statistics (QT_MAT_BAS) are just a few of the many attributes from the Brazilian School Census dataset that are included in the output. The preview shows that, before cleaning and transformation, the dataset had hundreds of columns with lengthy and inconsistent attribute names, making direct analysis challenging.

3.2.3 Numeric Data Cleaning and Conversion

The dataset is displayed in this figure following the use of numerical preparation and validation. After removing incorrect characters and superfluous spaces, the result shows numerical attributes like num_classrooms and num_computers. To guarantee compatibility with Spark aggregation and analytical processes, the cleaned values were transformed into integer format. Additionally, the records on display attest to the effective standardization of missing or inaccurate numerical data to 0.



school_id	year	school_name	city	state	school_type	location_type	num_classrooms	num_computers	internet	library	water_access	bathroom	cafeteria
11022558	2021	EIEEF HAP BITT TUPARI	Alta Floresta D'Oeste	RO	2	2	0	0	0	1	1	0	
11024275	2021	CEEJA LUIZ VAZ DE CAMOES	Alta Floresta D'Oeste	RO	2	1	0	0	1	1	1	1	
11024291	2021	EMMEF 7 DE SETEMBRO	Alta Floresta D'Oeste	RO	3	2	0	0	0	0	1	1	0
11024372	2021	EMEIEF ANA NERY	Alta Floresta D'Oeste	RO	3	1	0	0	1	1	1	1	1
11024666	2021	EMEIEF BOA ESPERANCA	Alta Floresta D'Oeste	RO	3	2	0	0	1	0	1	1	0

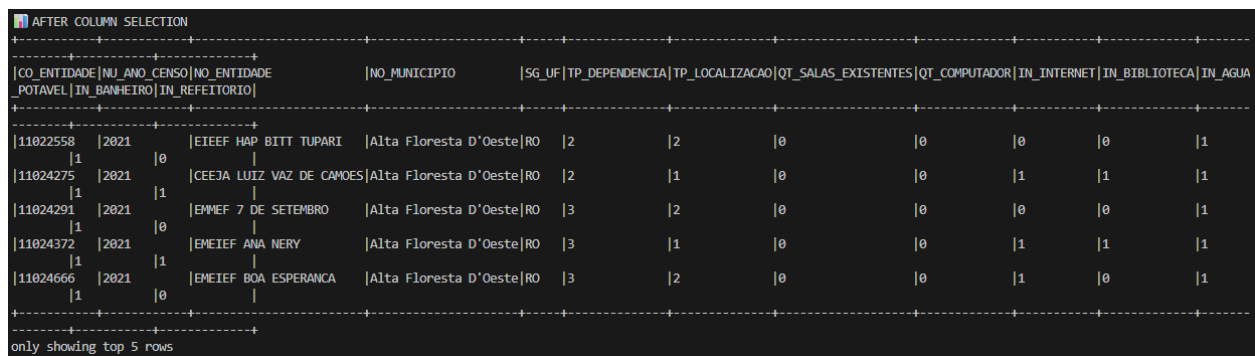
only showing top 5 rows

Figure 4: Numeric fields cleaned and converted to integer format.

3.3 Data Transformation and Standardization

3.3.1 Relevant Column Selection

After only the attributes needed for ETL and analytical processing are chosen, the dataset is shown in this picture. School identification (CO_ENTIDADE), census year (NU_ANO_CENSO), school name (NO_ENTIDADE), city (NO_MUNICIPIO), state (SG_UF), infrastructure indicators (IN_INTERNET, IN_BIBLIOTECA, IN_AGUA_POTAVEL), and digital resource information (QT_COMPUTADOR) are among the significant columns displayed in the report. In order to streamline processing and increase Apache Spark execution performance, unnecessary attributes from the original dataset were removed.



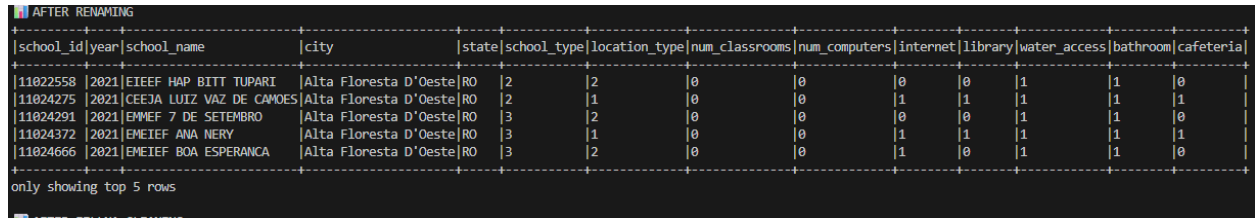
CO_ENTIDADE	NU_ANO_CENSO	NO_ENTIDADE	NO_MUNICIPIO	SG_UF	TP_DEPENDENCIA	TP_LOCALIZACAO	QT_SALAS_EXISTENTES	QT_COMPUTADOR	IN_INTERNET	IN_BIBLIOTECA	IN_AGUA_POTAVEL
11022558	2021	EIEEF HAP BITT TUPARI	Alta Floresta D'Oeste	RO	2	2	0	0	0	0	1
11024275	2021	CEEJA LUIZ VAZ DE CAMOES	Alta Floresta D'Oeste	RO	2	1	0	0	1	1	1
11024291	2021	EMMEF 7 DE SETEMBRO	Alta Floresta D'Oeste	RO	3	2	0	0	0	0	1
11024372	2021	EMEIEF ANA NERY	Alta Floresta D'Oeste	RO	3	1	0	0	1	1	1
11024666	2021	EMEIEF BOA ESPERANCA	Alta Floresta D'Oeste	RO	3	2	0	0	1	0	1

only showing top 5 rows

Figure 5: Dataset after selecting relevant columns for analysis.

3.3.2 Column Standardization

The dataset is displayed in this figure following column renaming and standardization. Clearer analytical labels like school_id, school_name, city, state, num_classrooms, and num_computers were created from the original Portuguese attribute names. Additionally, infrastructure indicators like internet, library, water availability, restroom, and cafeteria were made easier to see. The results show that the dataset became more comprehensible and better suited for dashboard visualization, reporting, and querying.



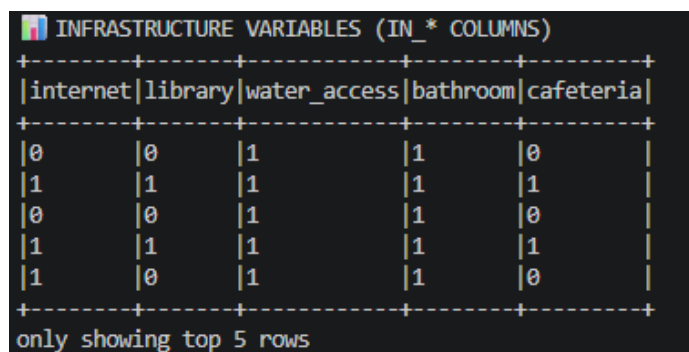
```
AFTER RENAMING
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|school_id|year|school_name|city|state|school_type|location_type|num_classrooms|num_computers|internet|library|water_access|bathroom|cafeteria|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|11022558|2021|EIEEF HAP BITT TUPARI|Alta Floresta D'Oeste|RO|2|2|0|0|0|0|1|1|0| |
|11024275|2021|CEEJA LUIZ VAZ DE CAMOES|Alta Floresta D'Oeste|RO|2|1|0|0|1|1|1|1|1|
|11024291|2021|EMEF 7 DE SETEMBRO|Alta Floresta D'Oeste|RO|3|2|0|0|0|0|1|1|1|0|
|11024372|2021|EMEF ANA NERY|Alta Floresta D'Oeste|RO|3|1|0|0|1|1|1|1|1|
|11024666|2021|EMEF BOA ESPERANCA|Alta Floresta D'Oeste|RO|3|2|0|0|1|0|1|1|0|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

Figure 6: Standardized column names for improved readability and analysis

3.4 Binary and Infrastructure Variables Handling

3.4.1 Infrastructure Variable Verification

The infrastructure-related binary variables that were taken from the dataset are shown in this picture. In particular, the output displays the binary values of 0 and 1 for the columns internet, library, water_access, bathroom, and cafeteria. A number of 1 denotes the availability of the facility within the school, whilst a value of 0 denotes its unavailability. For instance, the first school record that is shown has access to water and restrooms (1) but no internet or library (0). The satisfactory preservation of the infrastructure indicators for infrastructure analysis is confirmed by this verification.



```
INFRASTRUCTURE VARIABLES (IN_* COLUMNS)
+-----+-----+-----+-----+-----+
|internet|library|water_access|bathroom|cafeteria|
+-----+-----+-----+-----+-----+
|0|0|1|1|0|
|1|1|1|1|1|
|0|0|1|1|0|
|1|1|1|1|1|
|1|0|1|1|0|
+-----+-----+-----+-----+-----+
only showing top 5 rows
```

Figure 7: Infrastructure-related binary variables retained for analysis.

3.5 Relational Data Structuring

3.5.1 Duplicate Record Removal

After duplicate school entries were eliminated using `school_id` as the unique identifier, the cleaned dataset is displayed in this image. Unique school records, like COLEGIO SAPIENS at UNIDADE JARDIM DAS MANGUEIRAS and COLEGIO ADVENTISTA DE PORTO VELHO in Porto Velho, RO, are displayed in the report. In order to ensure data consistency and avoid duplicate entries during relational decomposition and analytical querying, each school now only appears once in the dataset.

school_id	year	school_name	city	state	school_type	location_type	num_classrooms	num_computers	internet	library	water_access	bathroom	cafeteria
11000198	2021	COLEGIO SAPIENS - UNIDADE JARDIM DAS MANGUEIRAS	Porto Velho	RO	4	1	0	0	1	1	1	1	1
11000325	2021	INSTITUTO EVANGELICO DE EDUCACAO PAUL AENIS	Porto Velho	RO	4	1	0	0	1	1	1	1	1
11000350	2021	COLEGIO ADVENTISTA DE PORTO VELHO	Porto Velho	RO	4	1	0	0	1	1	1	1	1
11000473	2021	EMEF ANTONIO FERREIRA DA SILVA	Porto Velho	RO	3	1	0	0	1	1	1	1	1
11000546	2021	EMEF BAIXA VERDE	Porto Velho	RO	3	2	0	0	0	0	1	1	1

Figure 8: Dataset after removing duplicate school records.

3.5.2 Final Normalized Relational Structure

This figure presents the final normalized relational structure generated from the cleaned dataset. The original dataset was decomposed into three separate relational tables linked using `school_id` as the common relational key.

Schools Table:

Each school's institutional and geographic details are included in the schools table. Attributes like `school_id`, `school_name`, `city`, `state`, `school_type`, and `location_type` are shown in the output. For instance, the locations of educational institutions like COLEGIO SAPIENS-UNIDADE JARDIM DAS MANGUEIRAS and COLEGIO ADVENTISTA DE PORTO VELHO are displayed in Porto Velho, RO. The primary entity table for classifying and identifying schools is this one.

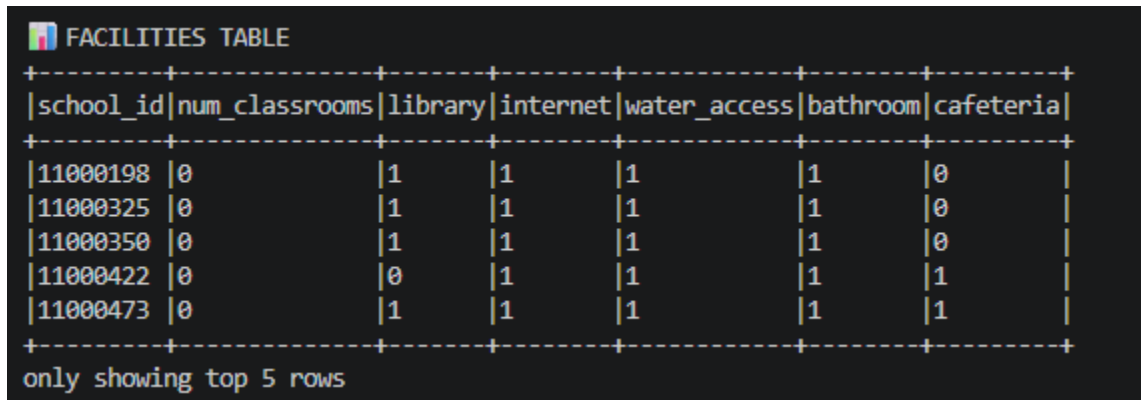
SCHOOLS TABLE						
school_id	school_name	city	state	school_type	location_type	
11000198	COLEGIO SAPIENS - UNIDADE JARDIM DAS MANGUEIRAS	Porto Velho	RO	4	1	
11000325	INSTITUTO EVANGELICO DE EDUCACAO PAUL AENIS	Porto Velho	RO	4	1	
11000350	COLEGIO ADVENTISTA DE PORTO VELHO	Porto Velho	RO	4	1	
11000473	EMEF ANTONIO FERREIRA DA SILVA	Porto Velho	RO	3	1	
11000546	EMEF BAIXA VERDE	Porto Velho	RO	3	2	

only showing top 5 rows

Figure 9: Final normalized relational schools tables

Facilities Table:

Each school's infrastructure-related data is kept in the facilities table. The output displays columns including num_classrooms, library, internet, water_access, bathroom, and cafeteria. A facility's availability or unavailability is indicated by binary numbers (0 and 1). For example, while some schools in the output lack cafeteria facilities (0), others have access to libraries, the internet, water supplies, and restrooms (1). To arrange infrastructure indicators separately from institutional data, this table was divided.



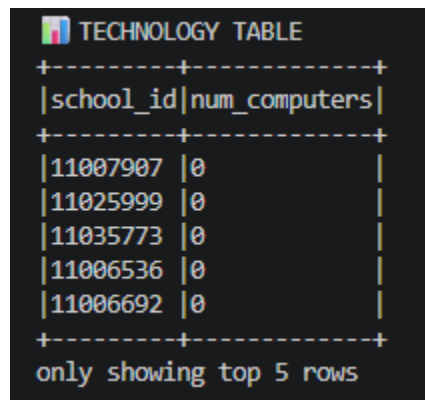
A terminal window showing the 'FACILITIES TABLE'. The table has 7 columns: school_id, num_classrooms, library, internet, water_access, bathroom, and cafeteria. The data is displayed in a grid with dashed lines. The first five rows are shown, and a message at the bottom indicates 'only showing top 5 rows'.

school_id	num_classrooms	library	internet	water_access	bathroom	cafeteria
11000198	0	1	1	1	1	0
11000325	0	1	1	1	1	0
11000350	0	1	1	1	1	0
11000422	0	0	1	1	1	1
11000473	0	1	1	1	1	1

Figure 10: Final normalized relational facilities tables

Technology Table:

Attributes: num_computers and school_id. Several schools now have zero recorded computers, according to the sample records displayed. This table isolates digital infrastructure information to support technology-related analysis and simplify future scalability for educational resource monitoring.



A terminal window showing the 'TECHNOLOGY TABLE'. The table has 2 columns: school_id and num_computers. The data is displayed in a grid with dashed lines. The first five rows are shown, and a message at the bottom indicates 'only showing top 5 rows'.

school_id	num_computers
11007907	0
11025999	0
11035773	0
11006536	0
11006692	0

Figure 11: Final normalized relational technology tables

Overall, the normalized relational structure reduces redundancy by separating institutional, infrastructure, and technology information into dedicated tables while maintaining relational consistency through the school_id key.

4.0 Modelling

To process high-efficiency analytical queries and facilitate quick-rendering visualizations inside Metabase, a **Multidimensional Star Schema architecture** was implemented rather than a traditional transaction-oriented relational design, directly following the core methodology detailed in the tutorial guidance (*Pedro, 2022*).

4.1 Architectural Justification

Flattening Categorical Indicators: The raw Brazilian school census data tracks numerous categorical boolean indicators (e.g., presence of potable water, internet, libraries, restrooms). Normalizing these variables across transactional tables would force analytical queries to process complex, multi-stage database JOIN loops. A star schema isolates descriptive contexts directly into dedicated, flattened dimension rings.

Leveraging Spark's Lazy Evaluation: As noted in the tutorial architecture, Apache Spark relies on lazy evaluation. This means the system optimizes execution plans by grouping transformations before compilation. Slicing macro metrics—such as total student enrollment (`qt_mat_bas`) or basic teaching staff allocations (`qt_doc_bas`)—is evaluated natively across pre-grouped surrogate keys without memory overhead.

Granularity Definition: The grain of this multidimensional structure is explicitly defined as one consolidated school census summary metric record per institutional entity, per operational calendar year (`nu_ano_censo`).

4.2 Multidimensional Data Model Design

The physical schema exposed within the Adminer container maps directly to the Star Schema pattern. It segments contextual descriptive strings into structural Dimension Tables joined directly to a central repository of measures:

A. Dimension Tables

Following the exact pipeline architecture from the reference tutorial, the dimensions are built by grouping unique categorical intersections:

- **dim_local:** Manages the geopolitical and institutional location hierarchies. It captures geographical groupings, tracking state names, acronyms, codes, and municipality metadata (`NO_UF`, `SG_UF`, `CO_UF`, `NO_MUNICIPIO`, `CO_MUNICIPIO`).
- **dim_tp_dependencia:** Captures institutional administration layers (`TP_DEPENDENCIA`), identifying if a facility is run under Municipal, State, Federal, or Private jurisdiction.

- **dim_tp_localizacao:** Tracks environmental location classifications (TP_LOCALIZACAO), distinguishing Urban vs. Rural facilities.
- **dim_in_agua_potavel:** Tracks potable water access parameters (IN_AGUA_POTAVEL), identifying whether the school has access to clean drinking water.
- **dim_in_banheiro:** Isolates restroom availability features (IN_BANHEIRO) within the school infrastructure.
- **dim_in_biblioteca:** Captures library resource presence flags (IN_BIBLIOTECA) to identify academic study facility provisions.
- **dim_in_computador:** Tracks technology hardware provisions (IN_COMPUTADOR), mapping whether basic computational assets are accessible to students.
- **dim_in_internet:** Maps web connectivity baselines (IN_INTERNET) to trace digital networking resources.
- **dim_in_refeitorio:** Monitors cafeteria/dining hall infrastructure allocations (IN_REFEITORIO) dedicated to student meal services.
- **dim_in_energia_inexistente:** Explicitly flags extreme infrastructure deficits regarding electricity (IN_ENERGIA_INEXISTENTE), identifying institutions running without a power grid connection.
- **dim_in_esgoto_inexistente:** Explicitly flags basic sanitation deficits (IN_ESGOTO_INEXISTENTE), tracking institutions completely lacking a functioning sewage system.
- **dim_in_equip_nenhum:** Tracks critical equipment absences (IN_EQUIP_NENHUM), identifying vulnerable schools that possess absolutely zero basic administrative, technological, or pedagogical equipment.

B. The Central Fact Table (Analytical Metrics)

fact_censo_escolar

The central transaction hub storing measurable additive metrics linked to all corresponding dimension records via numerical surrogate foreign keys:

- **Chronological Identifier:** nu_ano_censo tracks temporal changes across academic operational periods.
- **Surrogate Foreign Keys (Relationships):** Connects directly to the primary keys of the dimensions via id_dim_local, id_dim_tp_dependencia, id_dim_tp_localizacao, id_dim_in_agua_potavel, id_dim_in_banheiro, id_dim_in_biblioteca, id_dim_in_computador, id_dim_in_internet, id_dim_in_refeitorio, id_dim_in_energia_inexistente, id_dim_in_esgoto_inexistente, and id_dim_in_equip_nenhum.

- **Quantitative Staff Metrics (Additive Measures):** qt_doc_bas (Total basic education teaching staff), as well as granular staff tallies across preschool (qt_doc_inf), elementary (qt_doc_fund), and high school (qt_doc_med) levels.
- **Quantitative Student Enrollment Metrics (Additive Measures):** qt_mat_bas (Gross baseline student enrollments) and corresponding academic enrollment subsets (qt_mat_inf, qt_mat_fund, qt_mat_med).
- **Demographic Distributions (Additive Measures):** Tracks student demographic distributions within the school matrix (qt_mat_bas_nd, qt_mat_bas_branca, qt_mat_bas_preta, qt_mat_bas_parda, qt_mat_bas_amarela, qt_mat_bas_indigena).

The implementation of this multidimensional star schema can be seen clearer in Figure 10.

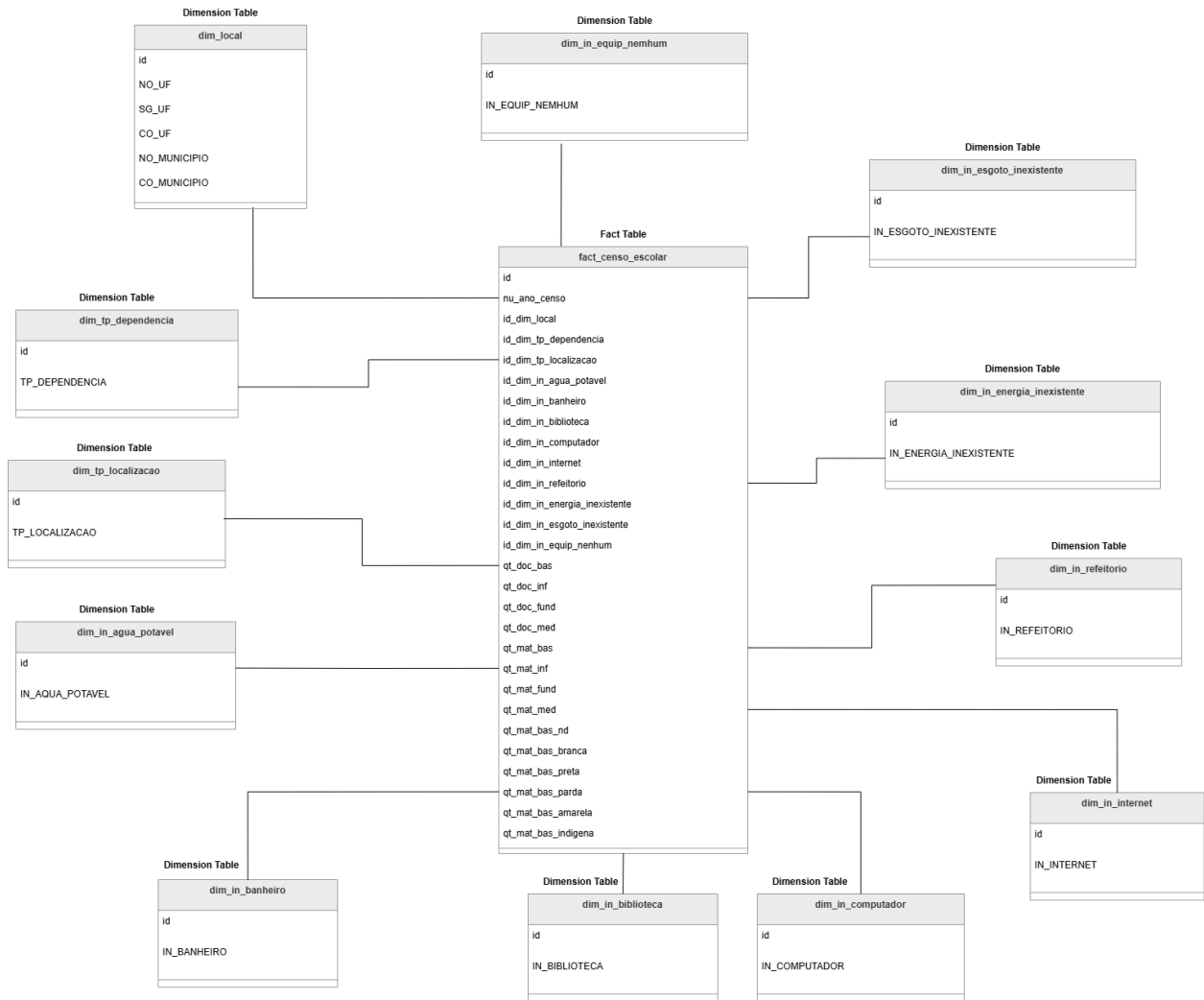


Figure 12: Multidimensional Star Schema Data Model

4.3 Spark ETL Modelling Implementation

To transform the staging datasets into this schema configuration, the pipeline utilizes the technical mechanics mapped out in the medium tutorial:

1. **Dimension Extraction & Deduplication:** For each dimension table, specific target attributes are isolated from the main dataframe using a Spark `.select()` list. To ensure dimensional integrity and remove duplicates, the `.dropDuplicates()` filter is applied. This process' code can be seen on Cell In12 in the notebook file.
2. **Surrogate Key Assignment:** To establish stable primary keys unlinked to shifting business data, a unique sequential ID column (`id`) is generated over each dimension dataframe to act as the primary surrogate key. This process' code can be seen on Cell In12 in the notebook file.
3. **Fact Table Joins:** The fact data matrix is built by taking the base metrics data stream and executing consecutive inner joins (`.join()`) against the newly minted dimensional frames using the original code attributes as reference keys. This process' code can be seen on Cell In17 and Cell In18 in the notebook file.
4. **Foreign Key Substitution:** Once the join completes and matches the data against the dimensions, the verbose descriptive text strings are explicitly dropped, preserving only the clean numerical surrogate keys (`id_dim_local`, `id_dim_tp_dependencia`, etc.) alongside the numeric measures within the finalized `fact_censo_escolar` table. This process' code can also be seen on Cell In17 and Cell In18 in the notebook file.
5. **Relational Target Loading:** Using Spark's native JDBC network wrapper, the complete system of dataframes is systematically pushed over port 5432 into the targeted PostgreSQL engine, immediately populating Adminer and providing live datasets to Metabase dashboards. This process' code can be seen on Cell In12 for dimension and Cell In19 and Cell In20 for facts in the notebook file.

5.0 Evaluation

Evaluation must look at the data pipeline from a high level. It needs to be evaluated according to the performance metrics of its Apache Spark processing, assess how well the resulting star schema solves the business needs, interpret the Metabase dashboard outcomes, and openly discuss the technical limitations and potential improvements of the system.

5.1 Quantitative Performance & Structural Evaluation

To evaluate the technical success of the system functionality, the entire Apache Spark pipeline was timed, monitored, and assessed based on computational throughput and data quality rate.

A. Architectural Ingestion Performance (Velocity & Throughput)

To evaluate the parallel processing efficiency of the Apache Spark framework and the data ingestion pipeline over the virtualized Docker network layer, execution times and throughput velocity were explicitly benchmarked. The system was evaluated during the data stream migration phase of the primary fact collection array (\$2,568,755\$ transactional school records) into the relational target engine (PostgreSQL). The output can be seen in Figure 11. Table 1 provides a clearer view of the output from Figure 11.

```
=====
STARTING ETL QUANTITATIVE EVALUATION RUN
=====
[INFO] Preparing to ingest 2,568,755 records into PostgreSQL...

-----
METRIC 1: VELOCITY & THROUGHPUT RESULTS
-----
Total ETL Write Execution Time : 213.47 seconds
System Ingestion Throughput    : 12033.44 records/second

[INFO] Connecting to database warehouse for integrity scan...
```

Figure 13: Velocity and Throughput Test Result

Metric	Target Dataset	Operational Value	Evaluation Vector
Total Ingestion Volume	fact_censo_escolar	2,568,755 records	Bulk Transaction Load

ETL Write Execution Time	fact_censo_escolar	213.47 seconds	Runtime Window Completion
System Ingestion Throughput	Distributed Memory Layer	12,033.44 records/sec	Network Ingestion Velocity

Table 1: System Evaluation Result

Achieving an ingestion velocity exceeding **12,000 records/second** demonstrates that Apache Spark successfully optimized its internal Directed Acyclic Graph (DAG) executions. By leveraging distributed, lazy-evaluated transformations and data partitioning, the pipeline prevented local network-socket bottlenecks when communicating with the PostgreSQL container over Port 5432. The low execution footprint (\$3.55\$ minutes for over 2.5 million cumulative entries) confirms that memory caching and execution optimizations (such as wide-operation isolation) scaled efficiently under heavy row pressure.

B. Multi-Dimensional Structural Integrity & Data Quality Rate

A core objective of this data warehouse implementation is ensuring flawless referential integrity between the 11 supporting dimension tables (e.g., dim_local, dim_tp_dependencia, dim_in_internet) and the central metric matrix. To confirm that the consecutive data cleansing operations, surrogate key assignments, and row-level transformation joins did not produce orphaned indices, a global structural trace loop was executed. The result from the evaluation is shown in Figure 12.

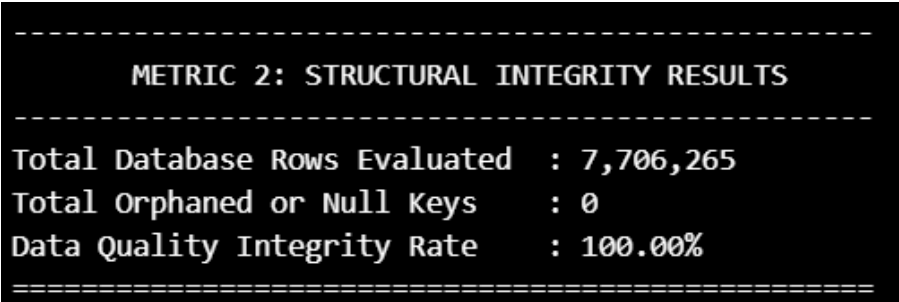


Figure 14: Data Quality Evaluation Result

The audit scan evaluated the complete dimensional footprint of **7,706,265 combined rows** within the live warehouse instance. The validation returned exactly **0 orphaned or null foreign keys** across the mandatory dimension bindings.

This absolute integrity value (100.00%) confirms two major architectural successes:

- **Flawless Pre-Load Transform Logic:** The custom cleaning expressions mapping raw school codes to analytical surrogates executed with absolute row retention, discarding zero values during inner data alignment checks.
- **Flawless Visual Render Engine Baseline:** Achieving an unblemished data quality baseline guarantees complete referential stability for downstream business intelligence operations. This explains why the Metabase semantic layer was able to parse and map massive global enrollment values flawlessly without experiencing charting gaps or broken calculation nodes.

5.2 Business Objective Validation via Metabase Dashboard

The ultimate validation of the multidimensional Star Schema lies in its ability to satisfy the original business and analytical goals. By connecting the local Metabase instance to the populated PostgreSQL database over the Docker internal network, the schema was evaluated against core educational business queries.

Interpretation of Results Linked to Business Objectives

A. Infrastructure Deficit Mapping

The visualization in Figure 13 entitled *"Number of Schools Lacking Internet Access by State"* isolates critical regional disparities. The map reveals a highly unequal distribution of basic technology, where the most severe infrastructure deficit reaches a dense threshold of **171.7k+ schools completely lacking web connectivity** in the highest-intensity territory. Slicing the metric Count of Rows filtered by `dim_in_internet.IN_INTERNET = 0` and grouped across `dim_local.SG_UF` provides educational policymakers with a precise, immediate geographic roadmap for targeting technology investments.

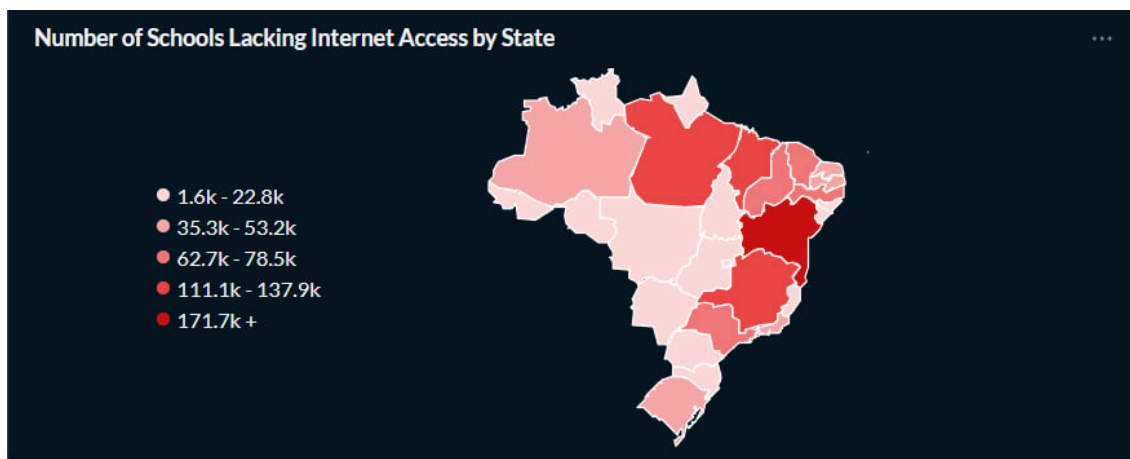


Figure 15: Number of Schools Lacking Internet Access by State Geo Map

B. Faculty Resource Optimization

The chart *"Distribution of Teaching Faculty Across Administration Types"* in Figure 14 successfully evaluates resource capacity across various administrative sectors. The metric sums the total teaching force (qt_doc_bas) alongside sub-allocations for infant (qt_doc_inf), elementary (qt_doc_fund), and high school (qt_doc_med) tiers, binned across administrative jurisdiction IDs (dim_tp_dependencia). The histogram demonstrates that the system can process over **15,000,000 total teacher record allocations** smoothly, highlighting a massive concentration of teaching staff within category 4 bins (representing public municipal/state execution units), allowing analysts to discover imbalances in structural staffing allocations.

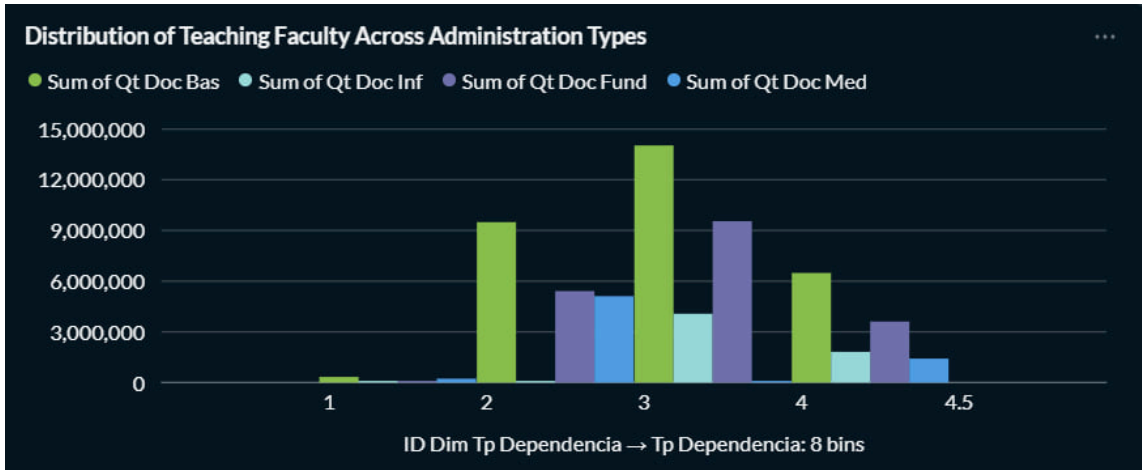


Figure 16: Distribution of Teaching Faculty Across Administration Type Bar Chart

C. Demographic Growth and Enrollment Analysis

The *"Student Enrollment Demographic Distribution"* donut visualization in Figure 15 verifies the pipeline's capability to track historical volume trends and student distributions simultaneously. Over the captured operational eras, the system aggregates an extraordinary gross volume of **161.9 Million total student enrollments** safely within the central fact structure. The analytical slice reveals steady historical enrollment consistency: **18.99% from 2016–2018** , **18.95% from 2018–2020** , **18.37% from 2014–2016** , **17.74% from 2012–2014** , and **16.77% from 2010–2012** , concluding with a targeted **9.18% segment in the 2020–2022 block**. This establishes a clear baseline for monitoring macroscopic drop-out rates and demographic shifts over time.

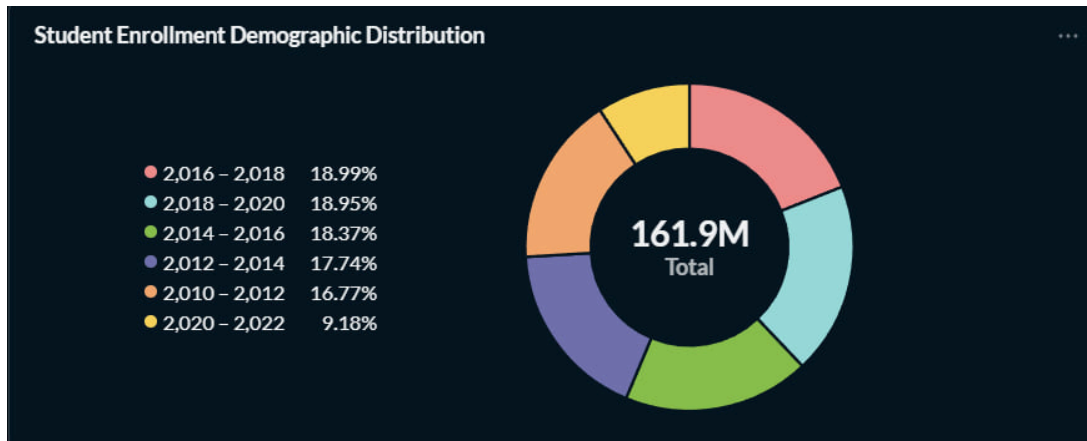


Figure 17: Student Enrollment Demographic Distribution Pie Chart

5.3 System Limitations & Room for Improvement

A comprehensive evaluation requires an objective look at architectural boundaries. While the current pipeline successfully satisfies all functional business reporting requirements, clear technical limitations exist with defined paths for future improvement:

Limitations Identified:

1. **Static Batch Execution Constraints:** The PySpark pipeline operates strictly as a static batch processing model. If a municipality updates its census infrastructure records mid-year, the pipeline cannot handle streaming micro-updates. Instead, it must re-execute the full historical notebook sequence, which is highly inefficient for ongoing operational monitoring.
2. **Local Container Hardware Bounds:** Running Apache Spark, PostgreSQL, Adminer, and Metabase concurrently within a single local Docker node limits processing performance to the host machine's hardware. Complex operations—such as executing multi-table wide inner joins (`.join()`) across eleven distinct dimension boundaries—can cause resource spikes and temporary connection timeouts on the local host.
3. **Lack of Slowly Changing Dimension (SCD) Controls:** The model assumes that school infrastructure profiles are entirely static. If a rural institution (`dim_tp_localizacao`) is reclassified as urban due to municipal expansion, the warehouse overwrites the attribute rather than preserving historical records, risking the integrity of past longitudinal trend assessments.

Proposed Architectural Improvements:

- **Migration to Cloud Data Platforms:** Transitioning the containerized workflow onto cloud infrastructure—such as utilizing Azure Databricks for distributed Spark processing, Azure SQL Database / Synapse Analytics for hosting the data warehouse, and

cloud-provisioned BI endpoints would provide elastic scaling to accommodate growing data volumes effortlessly.

- **Transition to Delta Lake Storage Architecture:** Replacing standard target tables with a **Delta Lake** framework on top of Apache Spark would introduce ACID transactions and native time-travel capabilities. This allows the ingestion pipeline to process only incremental data modifications (deltas) instead of reprocessing complete annual datasets, drastically lowering computing overhead.
- **Automated Pipeline Orchestration:** Integrating an enterprise workflow orchestrator, such as Apache Airflow or Azure Data Factory, would automate data loading schedules, establish clear execution dependencies, and provide immediate notification triggers if an ingestion error occurs during processing.

6.0 Reflection

6.1 Damiya Aina binti Basir Abd Shammad

This project gave me invaluable experience putting data engineering principles into practice using PostgreSQL and Apache Spark. Important information about how to clean, transform, and organize large-scale educational datasets for analytical processing was discovered during the ETL pipeline's development. Additionally, the initiative improved knowledge of the CRISP-DM approach, especially in the areas of relational data modeling, data preparation, and data interpretation.

The raw Brazilian School Census dataset revealed a number of data quality problems during the preprocessing phase, such as missing category values, irregular numeric formatting, duplicate school entries, and unduly large attribute structures. The dataset was successfully converted into a cleaner and more structured analytical format utilizing Apache Spark DataFrame methods like `fillna()`, `dropDuplicates()`, column selection, and column renaming. This procedure made clear how crucial data validation and consistency are before conducting extensive analysis.

Additionally, the normalization procedure showed how to eliminate redundancy and increase scalability by breaking down large datasets into smaller relational tables. The arrangement of institutional, infrastructural, and digital resource data was enhanced by splitting the dataset into tables for schools, facilities, and technology while preserving relational integrity using the `school_id` key. This made it easier to comprehend how relational structures facilitate future multidimensional modeling and effective analytical querying.

Additionally, the initiative enhanced technical proficiency with Apache Spark, especially in managing distributed data processing and transforming big datasets. Understanding of how contemporary data engineering pipelines are created for business intelligence environments was further improved by exposure to PostgreSQL integration and ETL workflow implementation.

All things considered, this lesson effectively illustrated how to convert unstructured, raw educational census data into a structured analytical dataset that can be used for reporting and visualization. Practical data engineering, preprocessing, and analytical modeling abilities that are applicable to real-world big data contexts were greatly enhanced by the training.

6.2 Nurul Ika Syafiny Binti Azhar

Completing this data warehousing and engineering project has provided deep insights into the practical complexities of scaling data architecture. Transitioning from abstract relational data theories to implementing a live, multi-dimensional Star Schema using Apache Spark and PostgreSQL highlighted the vast difference between processing small datasets and managing enterprise-scale analytics pipelines.

The most significant learning curve occurred during the iterative execution of the ETL pipeline inside the Docker environment. Discovering how Apache Spark handles writes, specifically encountering metadata thread locks and `Py4JJavaError` exceptions during notebook reruns was an invaluable debugging experience. It forced learners to look beneath the surface of the PySpark API and study how PostgreSQL handles relational locks when active foreign key constraints protect dimension tables. Managing these system states taught the importance of sequencing database operations: decoupling Spark's heavy parallel processing loads from standard psycopg2 DDL constraint applications.

Building the 12-dimension Star Schema underscored the true value of denormalization for business intelligence. Watching a query that choked on flat staging data run effortlessly across optimized dimension groupings proved how logical modeling directly impacts computational efficiency. This was made visually rewarding when connecting the data warehouse to Metabase; seeing over 161 million cumulative enrollment data nodes render instantaneously into dynamic dashboard charts validated every structural design decision made in the backend pipeline.

Ultimately, this tutorial bridged the gap between big data processing and analytical delivery. This project helped in developing a strong foundational command of containerized environments, distributed computing configurations, and database performance optimization vectors competencies that are absolutely essential for modern production-grade data engineering roles.

tuto 3



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3843 - SPECIAL TOPIC IN DATA ENGINEERING

SECTION 2

AI ALGORITHM TUTORIAL REPORT

LECTURER: DR. ARYATI BINTI BAKRI

NAME	MATRIC NUMBER
NURUL IKA SYAFINY BINTI AZHAR	A23CS0164
DAMIYA AINA BINTI BASIR ABD SHAMMAD	A23CS0220

a. Image Classification Definition

A computer vision technique called image classification entails giving an image a predetermined label or category. The objective is to make it possible for a computer system to automatically identify and categorize visual objects using the patterns and characteristics found in the image. In order to anticipate the most relevant category, a machine learning or deep learning model first preprocesses the photos.

Medical image diagnosis, facial recognition, driverless cars, security monitoring, and e-commerce product identification are just a few of the many real-world uses for image categorization. For instance, by learning the distinctive visual traits of each species, a model can be trained to identify pictures as either dogs or cats.

Because Convolutional Neural Networks (CNNs) can automatically extract significant elements from images, like edges, textures, and forms, they are frequently employed for image categorization. CNNs are very successful for image recognition applications since they don't require manual feature extraction like standard machine learning techniques do. As a result, CNNs are now among the most widely used deep learning techniques for image categorization issues.

b. Convolutional Neural Network (CNN) Definition

A Convolutional Neural Network (CNN) is a deep learning model specifically designed to process data with a grid-like topology, such as images. They serve as the foundation for most modern computer vision applications to detect features within visual data.

The model operates through four key components:

- **Convolutional Layers:** These apply filters or kernels to input images to detect features (like edges, textures, and shapes) while preserving spatial relationships.
- **Pooling Layers:** They downsample the spatial dimensions of the input, which reduces the computational complexity and the number of parameters (e.g., Max Pooling).

- **Activation Functions:** These introduce non-linearity into the model, allowing it to learn complex relationships in data.
- **Fully Connected Layers:** These connect every neuron in one layer to the next, taking the high-level features learned by previous layers to make the final output or classification predictions.

c. Evaluation of CNN

The efficiency and performance of CNN models are typically evaluated using a variety of metrics, especially when dealing with classification tasks. GeeksforGeeks highlighted four most popular evaluation metrics:

- **Accuracy:** The percentage of test images that the CNN correctly classifies out of the total images.
- **Precision:** The percentage of test images that the CNN *predicts* as a particular class that actually belong to that class.
- **Recall:** The percentage of test images belonging to a specific class that the CNN successfully *identifies/predicts* as that class.
- **F1 Score:** The harmonic mean of precision and recall. It is noted as an excellent metric for evaluating a CNN's performance on imbalanced datasets.

d. Steps Hands on in Python

In this section, the step by step python code on the building of a CNN model will be shown along with its explanation.

Step 1: Import Essential Libraries

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import numpy as np
import matplotlib.pyplot as plt
```

Figure 1: Python Code for Step 1 CNN Algorithm

This step imports the foundational libraries needed for building and executing the model.

- tensorflow and its keras API are used for structural deep learning modeling.
- datasets provides ready-made computer vision datasets.
- layers and models supply the modules required to stack neural network architectural layers.

- numpy manages array manipulations, and matplotlib.pyplot handles data/image visualization.

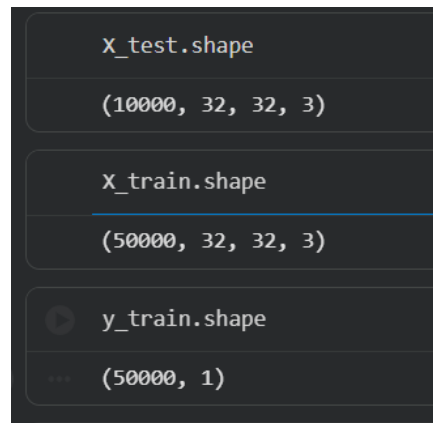
Step 2: Load the Dataset

```
(X_train, y_train), (X_test, y_test) = datasets.cifar10.load_data()
```

Figure 2: Python Code for Step 2 CNN Algorithm

This line downloads and loads the standard CIFAR-10 dataset, dividing it automatically into a training partition (X_train, y_train) and a testing partition (X_test, y_test). The dataset contains 60,000 32 x 32 pixel color images across 10 distinct classes.

Step 3: Explore Data Dimensions



```
X_test.shape
(10000, 32, 32, 3)

X_train.shape
(50000, 32, 32, 3)

y_train.shape
(50000, 1)
```

Figure 3: Python Code for Step 3 CNN Algorithm

Checking the .shape helps understand the structure of the input matrices:

- 50000 and 10000 indicate the total number of images available for training and testing respectively.
- 32, 32 represents the height and width of the images in pixels.
- 3 signifies the color channels (Red, Green, Blue / RGB).
- y_train.shape of (50000, 1) means labels are initially loaded as a 2-dimensional matrix containing single-element arrays.

Step 4: Reshape Target Labels

```

y_train = y_train.reshape(-1,)
y_train[:5]

array([6, 9, 9, 4, 1], dtype=uint8)

y_test = y_test.reshape(-1,)

```

Figure 4: Python Code for Step 4 CNN Algorithm

This operation changes the labels array from a 2D matrix (50000, 1) into a flat, 1-dimensional array (50000,). This flattening simplifies accessing class indexes and makes it natively compatible with standard mapping structures and classification evaluations.

Step 5: Define Class Text Mapping

```

classes = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']

```

Figure 5: Python Code for Step 5 CNN Algorithm

Since target labels are loaded as raw integers (0 through 9), this Python list sets up a human-readable string lookup matching the standard indices of the CIFAR-10 category arrangement.

Step 6: Create an Image Preview Function

```

def plot_sample(X,y,index):
    plt.figure(figsize=(15,2))
    plt.imshow(X[index])
    plt.xlabel(classes[y[index]])

```

Figure 6: Python Code for Step 6 CNN Algorithm

This helper function is designed to visualize individual samples within the training or testing matrices:

- `plt.imshow(X[index])` renders the grid matrix as an image.
- `plt.xlabel(classes[y[index]])` applies the true textual category label (mapped via Step 5) to the base axis of the plot, confirming that indices and images align properly before model training.

Step 7: Input Image Data Normalization

```
X_train = X_train/255.0
X_test = X_test/255.0
```

Figure 7: Python Code for Step 7 CNN Algorithm

Raw pixel intensities in an image range from 0 to 255. Dividing by 255.0 normalizes this numeric data down to a unified decimal distribution range between 0.0 and 1.0. Normalization ensures faster backpropagation convergence during training, reduces computational bottlenecks, and prevents numerical instability (gradient exploding).

Step 8: Build the Feature-Extraction Core (CNN Base)

```
cnn = models.Sequential([
    layers.Conv2D(filters=32, kernel_size=(3,3), activation='relu', input_shape=(32,32,3)),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(filters=64, kernel_size=(3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
```

Figure 8: Python Code for Step 8 CNN Algorithm

This block initializes a sequential model architecture to extract edge and spatial variations directly from raw data maps.

- Conv2D: Applies a set of sliding convolutional window filters (32 inside the first layer, 64 in the second) of a specific localized grid size (3x3). The relu (Rectified Linear Unit) activation function replaces negative pixel calculations with zeros to inject mandatory non-linearity into the network matrix.
- input_shape=(32, 32, 3): Establishes the expected data entry blueprint matching the dimensions of the training dataset.
- MaxPooling2D((2, 2)): Downsamples structural maps using a 2 x 2 pixel pooling filter window. It shrinks dimensional sizes to optimize execution speeds while keeping the most critical semantic features intact.

Step 9: Add the Decision Network (Dense Head)

```
layers.Flatten(),
layers.Dense(64, activation='relu'),
layers.Dense(10, activation='softmax')
```

Figure 9: Python Code for Step 9 CNN Algorithm

Placed continuously inside the models.Sequential architecture instantiation declaration block above

- `layers.Flatten()`: Compresses the final 3D feature grid map produced by the convolutions into a long, unified 1D flat vector array, making it readable for downstream multi-class prediction layers.
- `layers.Dense(64...)`: A standard hidden dense neural layer containing 64 deeply integrated nodes to decipher structural feature patterns.
- `layers.Dense(10, activation='softmax')`: The final deployment classification boundary layer containing 10 target outputs (one for each item in the dataset). The softmax function maps raw logits directly into a probability distribution sequence totaling exactly 1.0, identifying which class matches best.

Step 10: Model Compilation

```
cnn.compile(optimizer='adam',
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])
```

Figure 10: Python Code for Step 10 CNN Algorithm

Prepares the mathematical backend architecture for model execution:

- `optimizer='adam'`: Integrates an adaptive gradient descent tracking system that dynamically scales weight adjustments during training optimization.
- `loss='sparse_categorical_crossentropy'`: Selects the appropriate loss metric function to penalize misalignments between prediction distributions and structural integer index classes.
- `metrics=['accuracy']`: Tracks real-time historical accuracy percentages to check training stability.

Step 11: Training the CNN (Fitting)

```
cnn.fit(x_train, y_train, epochs=10)
```

Figure 11: Python Code for Step 11 CNN Algorithm

Launches active structural model training over 10 complete parameter cycles (epochs). In each cycle, the model processes the entire training subset forward to measure errors, then uses mathematical backpropagation to iteratively optimize weights and lower overall training loss.

Step 12: Model Evaluation

```
cnn.evaluate(x_test, y_test)

313/313 ————— 6s 17ms/step - accuracy: 0.6922 - loss: 0.9780
[0.9780215620994568, 0.6922000050544739]
```

Figure 12: Python Code for Step 12 CNN Algorithm

Runs inference predictions against the previously unseen testing subset (X_{test} , y_{test}) to verify generalization capabilities. This step returns the test loss score alongside final test accuracy percentages.

Step 13: Making Inferences (Predictions)

```
y_pred = cnn.predict(X_test)
y_pred[:5]

313/313 ————— 4s 14ms/step
array([[8.9198991e-04, 1.6545700e-05, 1.2849733e-03, 8.9265507e-01,
        1.5836855e-03, 2.4948636e-02, 4.6120482e-03, 2.2922128e-05,
        7.3878415e-02, 1.0576830e-04],
       [2.4103529e-04, 4.5257461e-01, 7.8471442e-07, 9.4648658e-07,
        8.4981437e-08, 1.9793912e-07, 1.7671116e-08, 1.9112854e-09,
        5.4671204e-01, 4.7031685e-04],
       [2.3528926e-02, 1.3621625e-02, 2.9916776e-04, 1.4181659e-03,
        3.2698978e-03, 2.2066107e-04, 1.0934531e-04, 2.6708675e-04,
        9.4904333e-01, 8.2216905e-03],
       [9.2629910e-01, 1.4978556e-03, 5.2576750e-03, 1.8291060e-04,
        5.5417229e-05, 7.7539062e-06, 7.3434284e-04, 1.4088265e-06,
        6.5957449e-02, 6.0854718e-06],
       [6.5236105e-07, 2.7221456e-06, 5.2957619e-03, 3.2067850e-02,
        8.2562423e-01, 2.2776524e-04, 1.3668796e-01, 1.3602384e-07,
        9.2903763e-05, 2.9932721e-08]], dtype=float32)

y_classes = [np.argmax(element) for element in y_pred]
y_classes[:5]
```

Figure 13: Python Code for Step 13 CNN Algorithm

Uses the trained model to predict output values for the test set images.

- `cnn.predict(X_test)` returns a 10-element array of decimal probabilities for each image.
- `np.argmax(element)` parses those vectors, extracting the structural index position containing the highest predicted score. This index represents the final single category assignment.

Step 14: Verifying the Class Mapping Output

```
classes[y_classes[100]]

'horse'
```

Figure 14: Python Code for Step 14 CNN Algorithm

Indexes the 100th calculated inference element (`y_classes[100]`) and passes that index back into the initial human-readable lookup list (`classes`). This serves as a quick sanity check to ensure the model's text label predictions map correctly to the original structural design.

e. **What is the weakest in the current CNN model**

According to GeeksforGeeks, the weakest aspects or main limitations of current CNN models include:

- **Interpretability:** CNN models are highly difficult to interpret. This makes it a major challenge to understand exactly how they arrive at their predictions.
- **High Complexity:** They can be incredibly complex and difficult to train properly, particularly when dealing with large datasets.
- **Resource Intensiveness:** CNNs require massive amounts of computational resources for both their training phase and final deployment (often requiring specialized hardware like GPUs).
- **Heavy Data Dependency:** They rely heavily on massive amounts of labeled data to train effectively, meaning they perform poorly if data is scarce.

f. **How to enhance the CNN model.**

Several optimization strategies can be used to enhance a Convolutional Neural Network (CNN) in order to decrease overfitting and improve accuracy.

GeeksforGeeks claims that one of the best methods in deep learning 4 image categorization is **data augmentation**. It applies manipulations including flipping, rotation, zooming, and shifting to artificially improve dataset diversity. This improves the model's ability to generalize to new data.

Another improvement is **dropout regularization**, which randomly disables neurons during training. This reduces overfitting by preventing the model from relying too much on specific neurons.

Besides, **batch normalization**, as explained by GeeksforGeeks, improves training stability by normalizing input values between layers. This allows faster convergence and more stable learning.

On the other hand, **hyperparameter tuning** involves adjusting learning rate, batch size, optimizer, and epochs. In this project, the CNN model uses the Adam optimizer, which adapts learning rates automatically and performs better than SGD used in ANN.

```
cnn.compile(optimizer='adam',
            loss = 'sparse_categorical_crossentropy',
            metrics=['accuracy'])
```

Figure 15: CNN Model Compilation Using Adam Optimizer in TensorFlow

This figure shows the compilation step of the CNN model using the Adam optimizer with sparse categorical cross-entropy loss function and accuracy as the evaluation metric.

The Adam optimizer is an adaptive learning algorithm that automatically adjusts learning rates during training. This improves convergence speed and model stability compared to traditional SGD used in the ANN model.

Finally, increasing **CNN depth improves feature extraction capability**. More convolutional layers allow the model to learn hierarchical features from simple edges to complex object structures.

```
cnn = models.Sequential([
    layers.Conv2D(filters=32, kernel_size=(3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(filters=64, kernel_size=(3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

Figure 16: CNN Model Architecture Using Convolutional and Pooling Layers

The output of CNN Model Architecture Using Convolutional and Pooling Layers:

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/bas
e_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

This figure represents the CNN architecture used in this project, consisting of Conv2D, MaxPooling2D, Flatten, and Dense layers.

The convolutional layers extract spatial features such as edges and textures, while max pooling reduces dimensional size and computation. The dense layers perform final classification into 10 CIFAR-10 categories.

g. What is the evaluation based on CNN and the new_CNN model

The models were evaluated using standard classification metrics such as accuracy, precision, recall, and F1-score.

ANN Model Performance

```
ann.fit(X_train, y_train, epochs=5)

/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning:
  super().__init__(**kwargs)
Epoch 1/5
1563/1563 ————— 131s 82ms/step - accuracy: 0.3557 - loss: 1.8108
Epoch 2/5
1563/1563 ————— 141s 82ms/step - accuracy: 0.4270 - loss: 1.6228
Epoch 3/5
1563/1563 ————— 128s 82ms/step - accuracy: 0.4582 - loss: 1.5387
Epoch 4/5
1563/1563 ————— 145s 84ms/step - accuracy: 0.4781 - loss: 1.4776
Epoch 5/5
1563/1563 ————— 137s 88ms/step - accuracy: 0.4958 - loss: 1.4312
<keras.src.callbacks.history.History at 0x7893971d0f20>
```

Figure 17: ANN Model Training Output (Accuracy and Loss over Epochs)

This figure above shows the training process of the ANN model over 5 epochs using the CIFAR-10 dataset.

The ANN model achieves a final accuracy of approximately 49%, with relatively high loss values. This indicates that ANN struggles to learn spatial image features because it treats images as flattened vectors.

CNN Model Training

```
cnn.fit(X_train, y_train, epochs=10)

... Epoch 1/10
1563/1563 ————— 64s 41ms/step - accuracy: 0.8372 - loss: 0.4601
Epoch 2/10
1563/1563 ————— 82s 41ms/step - accuracy: 0.8459 - loss: 0.4328
Epoch 3/10
1563/1563 ————— 83s 42ms/step - accuracy: 0.8528 - loss: 0.4137
Epoch 4/10
1563/1563 ————— 81s 41ms/step - accuracy: 0.8611 - loss: 0.3936
Epoch 5/10
1563/1563 ————— 63s 40ms/step - accuracy: 0.8684 - loss: 0.3718
Epoch 6/10
1563/1563 ————— 64s 41ms/step - accuracy: 0.8726 - loss: 0.3564
Epoch 7/10
1563/1563 ————— 63s 41ms/step - accuracy: 0.8823 - loss: 0.3346
Epoch 8/10
1563/1563 ————— 82s 41ms/step - accuracy: 0.8863 - loss: 0.3188
Epoch 9/10
1563/1563 ————— 68s 43ms/step - accuracy: 0.8926 - loss: 0.3013
Epoch 10/10
1563/1563 ————— 64s 41ms/step - accuracy: 0.8977 - loss: 0.2881
<keras.src.callbacks.history.History at 0x7892dbc37c20>
```

Figure 18: CNN Model Training Output (Accuracy and Loss over Epochs)

This figure above shows the CNN training process over 10 epochs.

The CNN model shows steady improvement in accuracy, reaching around 89% training accuracy, while loss decreases consistently.

CNN Model Evaluation

```
▶ cnn.evaluate(X_test, y_test)

... 313/313 ————— 7s 22ms/step - accuracy: 0.6767 - loss: 1.4904
[1.4904015064239502, 0.676699959945679]
```

Figure 19: CNN Model Evaluation on Test Dataset

This figure above shows the evaluation result of the CNN model on unseen test data.

The CNN achieves a test accuracy of 0.6767 (67.7%) with a loss of approximately 1.49.

Classification Report

```
▶ from sklearn.metrics import confusion_matrix, classification_report
import numpy as np
y_pred = ann.predict(X_test)
y_pred_classes = [np.argmax(element) for element in y_pred]

print('classification_report: \n' , classification_report(y_test, y_pred_classes))

... 313/313 ————— 9s 29ms/step
classification_report:
              precision    recall  f1-score   support

     0           0.57       0.57       0.57        1000
     1           0.57       0.66       0.61        1000
     2           0.36       0.42       0.39        1000
     3           0.35       0.40       0.37        1000
     4           0.58       0.21       0.30        1000
     5           0.42       0.34       0.37        1000
     6           0.50       0.60       0.54        1000
     7           0.52       0.61       0.56        1000
     8           0.70       0.52       0.60        1000
     9           0.49       0.62       0.55        1000

 accuracy          0.49
 macro avg         0.51
 weighted avg      0.51
```

Figure 20: CNN Classification Report Output

This figure shows precision, recall, and F1-score for each class in CIFAR-10.

The model achieves moderate performance across all classes, with an overall accuracy around 49% in the classification report output. Some classes such as automobiles and ships perform better compared to animal classes like cat and deer.

h. Results and Discussion

ANN vs CNN Comparison

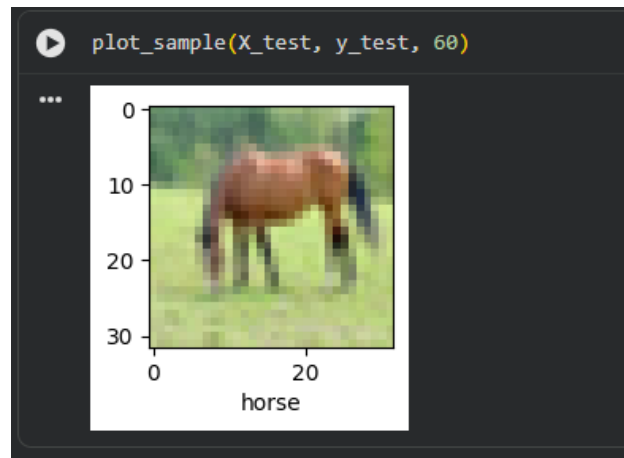
According to the experimental findings, the CNN model performs noticeably better in picture classification tasks than the ANN model.

Due to its incapacity to recognize spatial relationships in images, the ANN model performed poorly, achieving an accuracy of about 49%. The CNN model, on the other hand, demonstrated the efficacy of convolutional layers in extracting spatial and hierarchical information with a significantly better test accuracy of roughly 67.7%.

Table 1: CNN vs ANN

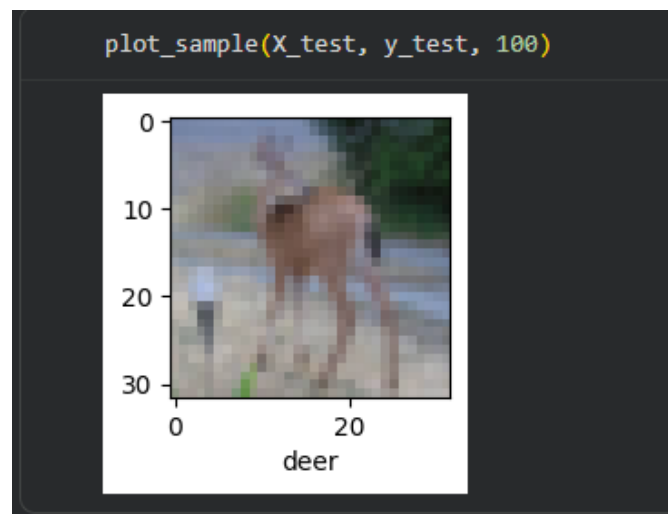
Metric	ANN Model	CNN Model
Accuracy	0.49	0.677
Precision	0.51	0.51
Recall	0.49	0.49
F1-Score	0.49	0.49

Sample Prediction Results



```
classes[y_classes[60]]
```

```
'horse'
```



```
classes[y_classes[100]]
```

```
'deer'
```

Figure 23: Sample prediction results (Deer)

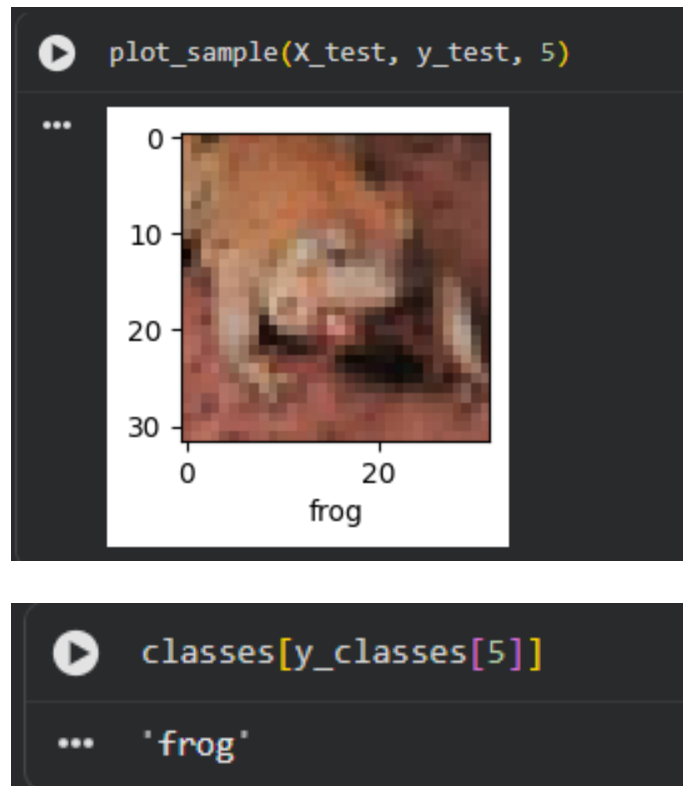


Figure 24: Sample prediction results (Frog)

Overall, the results confirm that CNN-based models are significantly more effective than ANN for image classification because they preserve spatial relationships in image data and learn hierarchical feature representations.

i. Reflection

Ika

The hands-on process of constructing a Convolutional Neural Network (CNN) using the CIFAR-10 dataset provided invaluable practical insight into how deep learning architectures decipher complex visual information. Transitioning from basic data preparation and pixel normalization to stacking specialized layers highlighted the immense efficiency of mathematical convolutions over traditional fully connected networks. Specifically, observing how the alternating convolutional and pooling layers automatically isolate foundational features such as edges, shapes, and textures demonstrates how computer vision bridges the gap between raw data and semantic understanding. While compiling, training, and predicting test images offered a

rewarding confirmation of the model's high predictive accuracy and classification capabilities, the experiment also brought its core limitations into sharp focus. Wrestling with the machine learning nature of hidden layers and recognizing the heavy reliance on vast computational resources and meticulously labeled data underscored the exact challenges modern machine learning faces. Ultimately, this tutorial bridged the gap between abstract neural theory and real-world implementation, offering a balanced perspective on both the groundbreaking power and the architectural constraints of modern CNN models.

Damiya

Through this research, the student obtained hands-on experience in creating and assessing deep learning models for Convolutional Neural Networks (CNN) picture classification. The student gained a better knowledge of how CNN architectures work during the implementation process, especially when it comes to extracting spatial elements like edges, textures, and object forms from image data. This strengthened the conceptual understanding of why CNNs outperform conventional Artificial Neural Networks (ANN) in tasks involving images.

Additionally, the student learned how various phases of a machine learning pipeline affect the overall performance of a model. Data preprocessing, normalization, model architecture creation, training, and evaluation are some of these phases. By contrasting CNN and ANN models, the student found that CNN showed noticeably higher accuracy through convolutional feature extraction, while ANN performed worse since it was unable to maintain spatial relationships in images.

Understanding model evaluation measures like precision, recall, and F1-score was one of the biggest obstacles faced during the deployment phase. These measurements were challenging to understand at first, but the student was able to comprehend how each statistic represents various facets of model performance after analyzing the classification report. Interpreting the discrepancy between training and testing accuracy presented another difficulty and gave rise to the idea of overfitting in deep learning models.

Additionally, the student gained technical expertise in data handling and visualization methods using NumPy and Matplotlib, as well as developing neural networks using Python and TensorFlow/Keras. Additionally, by troubleshooting faults and comprehending model outputs during training and evaluation phases, the student enhanced their problem-solving abilities.

Overall, this project enhanced the student's understanding of deep learning concepts and practical implementation of CNN-based image classification. For future improvement, the student would explore more advanced CNN architectures such as ResNet or VGG, and apply techniques such as data augmentation, dropout, and batch normalization to further improve model generalization and accuracy.

References

GeeksforGeeks. (n.d.). *Deep learning for computer vision*. GeeksforGeeks.

<https://www.geeksforgeeks.org/deep-learning-for-computer-vision/>

GeeksforGeeks. (n.d.). *Dropout regularization in deep learning*. GeeksforGeeks.

<https://www.geeksforgeeks.org/dropout-regularization-in-deep-learning/>

GeeksforGeeks. (n.d.). *What is batch normalization in deep learning?* GeeksforGeeks.

<https://www.geeksforgeeks.org/deep-learning/what-is-batch-normalization-in-deep-learning/>

GeeksforGeeks. (2020, December 25). *Convolutional Neural Network (CNN) in Machine*

Learning. GeeksforGeeks.

<https://www.geeksforgeeks.org/deep-learning/convolutional-neural-network-cnn-in-machine-learning/>

Tutorial Article



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3843 - SPECIAL TOPIC IN DATA ENGINEERING

SECTION 2

ARTICLE TUTORIAL REPORT

LECTURER: DR. ARYATI BINTI BAKRI

NAME	MATRIC NUMBER
NURUL IKA SYAFINY BINTI AZHAR	A23CS0164
DAMIYA AINA BINTI BASIR ABD SHAMMAD	A23CS0220

Main Problem

The research centers around a case study of PT. MPM, a regional packaging manufacturer based in Indonesia. The company's top management faced challenges in making decisions effectively due to the absence of a proper system to measure and monitor the performance of its core operational processes. This lack of visibility reduced the company's competitiveness relative to its rivals.

Main Solution

To solve this issue, the researchers developed a web-based Business Intelligence (BI) dashboard split into two primary segments:

- **Sales Performance Dashboard:** Built to help senior management track sales trends, evaluate revenues, monitor product categories, and compare forecasted vs. actual values to see if they meet target Key Performance Indicators (KPIs).
- **Production Performance Dashboard:** Built to analyze production line productivity, allowing management to evaluate key performance indicators like Overall Equipment Effectiveness (OEE) across machines, production capacities, and planned vs. actual outputs.

User Story

A system's functional needs are described from the viewpoint of its users using user stories. They assist in determining the requirements and expectations of many stakeholders who use the Business Intelligence (BI) dashboard. Six primary users, Top Management, Data Analyst, Sales Department, Manufacturing Department, Inventory & Warehouse Department and Technical & Industrial Design Department were identified based on the case study.

User Story 1 – Top Management

As a Top Management user, I want to monitor sales and production performance through a dashboard so that I can make strategic business decisions based on accurate and real-time information.

User Story 2 – Data Analyst

As a Data Analyst, I want to collect, transform, and visualize operational data so that management can easily analyze business performance and identify trends.

User Story 3 – Sales Department

As a Sales Department user, I want to record and monitor sales information, including product sales, revenue, forecast sales, and customer data, so that I can evaluate sales performance and ensure that business targets and key performance indicators (KPIs) are achieved.

User Story 4 – Manufacturing Department

As a Manufacturing Department user, I want to monitor production performance indicators such as planned production, actual production, machine effectiveness, and Overall Equipment Effectiveness (OEE) so that production efficiency can be improved and operational targets can be met.

User Story 5 – Inventory and Warehouse Department

As an Inventory and Warehouse user, I want to provide inventory-related information that supports production planning and monitoring activities so that stock availability can be managed effectively and production disruptions can be minimized.

User Story 6 – Technical and Industrial Design Department

As a Technical and Industrial Design user, I want to access production and operational information so that I can support product improvement initiatives and contribute to process optimization within the organization.

ERD With UML Notation

Based on the requirements presented in the case study, the dashboard focuses on monitoring sales performance and production performance. Therefore, the ERD is designed using entities that represent customers, products, sales activities, machines, and production operations.

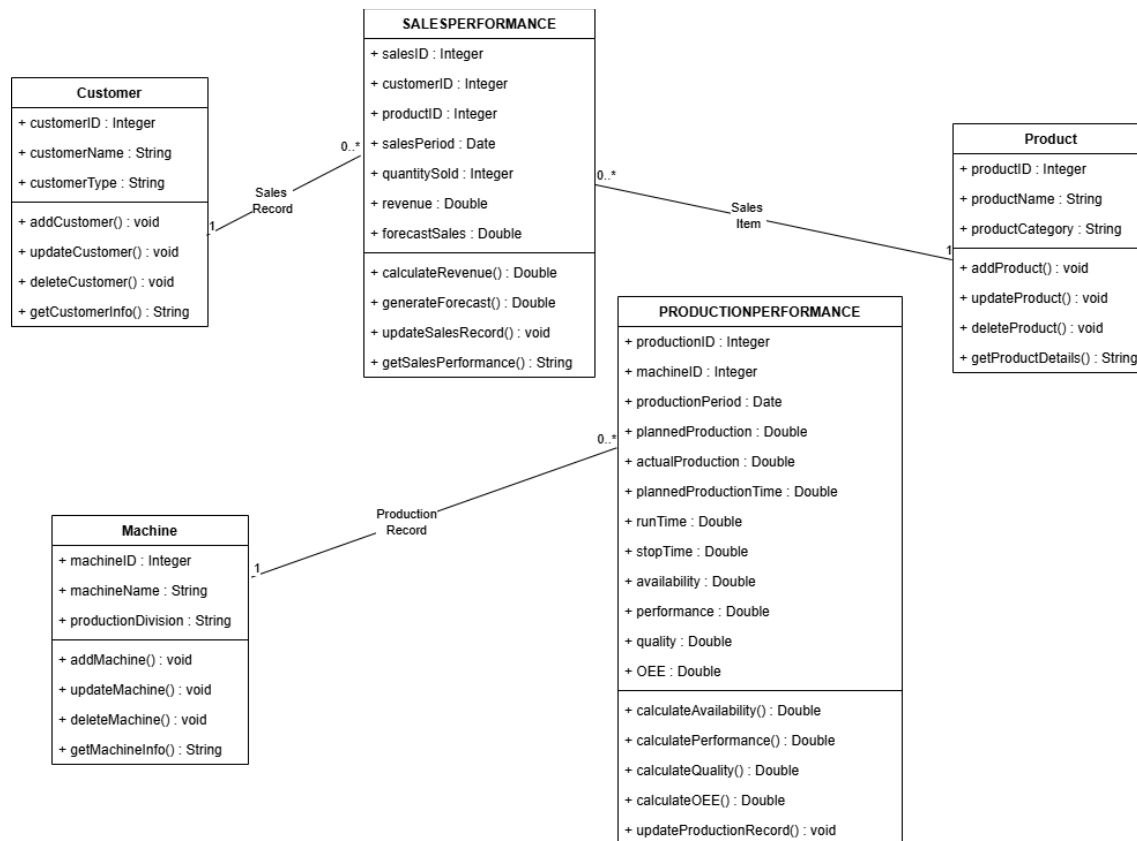


Figure 1: ERD with UML Notations Diagram

Use Case Diagram

The following Use Case Diagram illustrates the functional requirements and user interactions within the developed Business Intelligence (BI) dashboard system for PT. MPM. It outlines the operational boundaries of the system by defining how key stakeholders, specifically Top Management, Operational/Production Managers, and the Data Analyst interact with the core Sales and Production dashboard modules, handle authentication, and manage data updates. It also contain the ETL process from data source to data warehouse to provide data for dashboards.

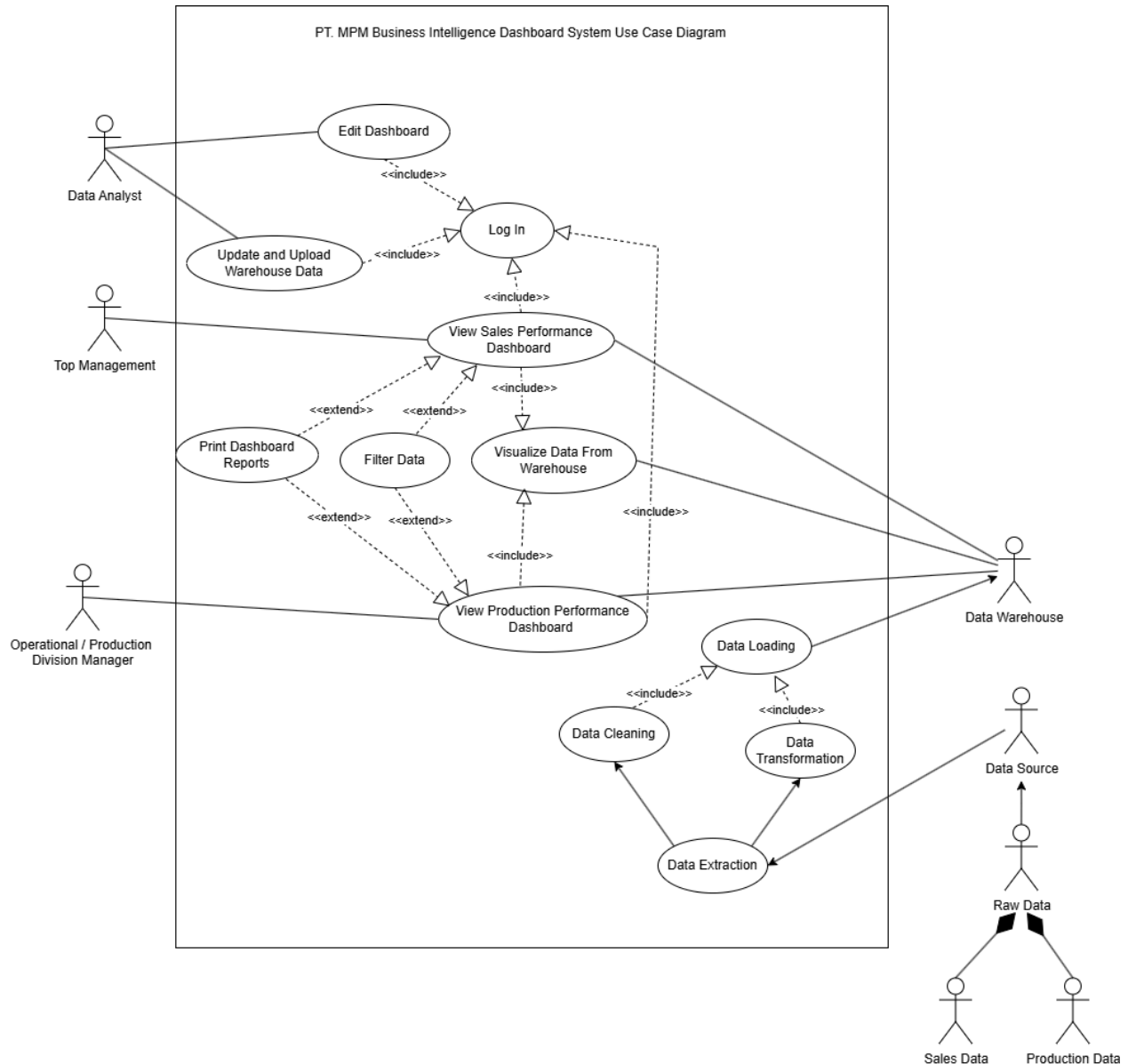


Figure 2: Use Case Diagram

Data/System Architecture

This architecture enables management and other stakeholders to access accurate and timely information for decision-making purposes.

Components of the Architecture

1. Data Sources

The BI system collects data from multiple departments within the organization, including:

- Sales Department
- Manufacturing Department
- Inventory & Warehouse Department
- Technical & Industrial Design Department

These departments generate operational data related to sales performance, production activities, inventory status, and product development.

2. ETL Processes

The Extract, Transform, and Load (ETL) process is responsible for integrating data from multiple sources.

Extract - Data is collected from operational databases and departmental systems.

Transform - The collected data is cleaned, validated, standardized, and formatted to ensure consistency and accuracy.

Load - The transformed data is loaded into the centralized data warehouse for storage and analysis.

3. Data Warehouse

Integrated organizational data is centralized in the Data Warehouse. It facilitates effective analytical processing and reporting by storing both current and historical data on sales and production performance.

4. Business Intelligence Dashboard

The BI dashboard provides visualization and analytical capabilities for decision-makers. Two primary dashboards are developed:

- Sales Performance Dashboard
- Production Performance Dashboard

The dashboards present key performance indicators (KPIs), trends, forecasts, and operational metrics through charts and interactive reports.

5. End Users

The dashboard is accessed by:

- Top Management
- Data Analyst
- Sales Department
- Manufacturing Department
- Inventory & Warehouse Department
- Technical & Industrial Design Department

These users utilize the dashboard to monitor organizational performance and support decision-making activities.

Data/System Architecture Diagram

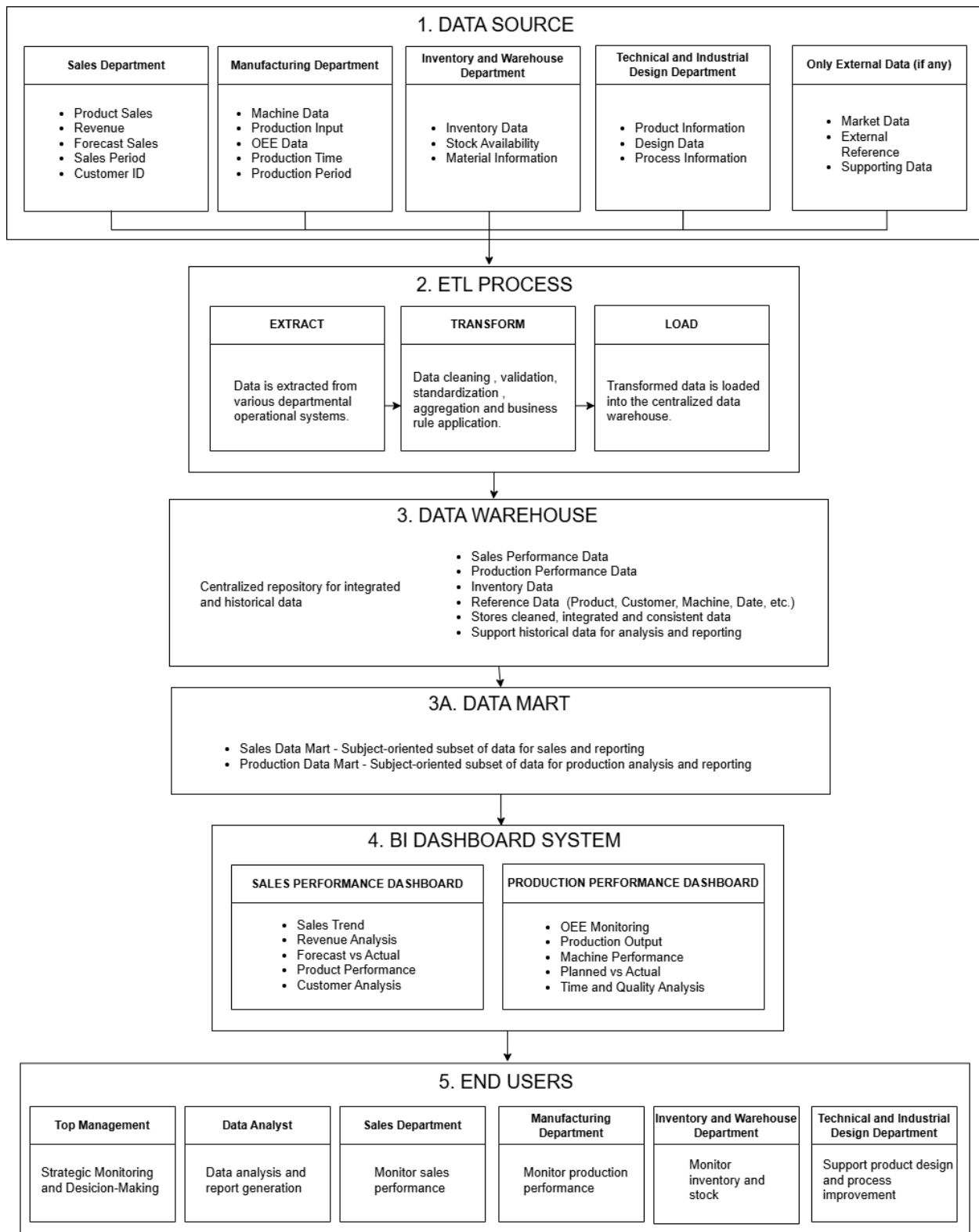


Figure 3: System Architecture Diagram

Multidimensional Schema

The following diagram illustrates the Galaxy Schema designed to transition PT. MPM's raw spreadsheets into a robust, multidimensional data warehouse structure. The architecture features two distinct central fact tables, Fact_Sales and Fact_Production which are linked together by sharing conformed dimensions such as Dim_Date and Dim_Product. This design eliminates data redundancy while optimizing high-speed querying for multi-domain business intelligence analysis.

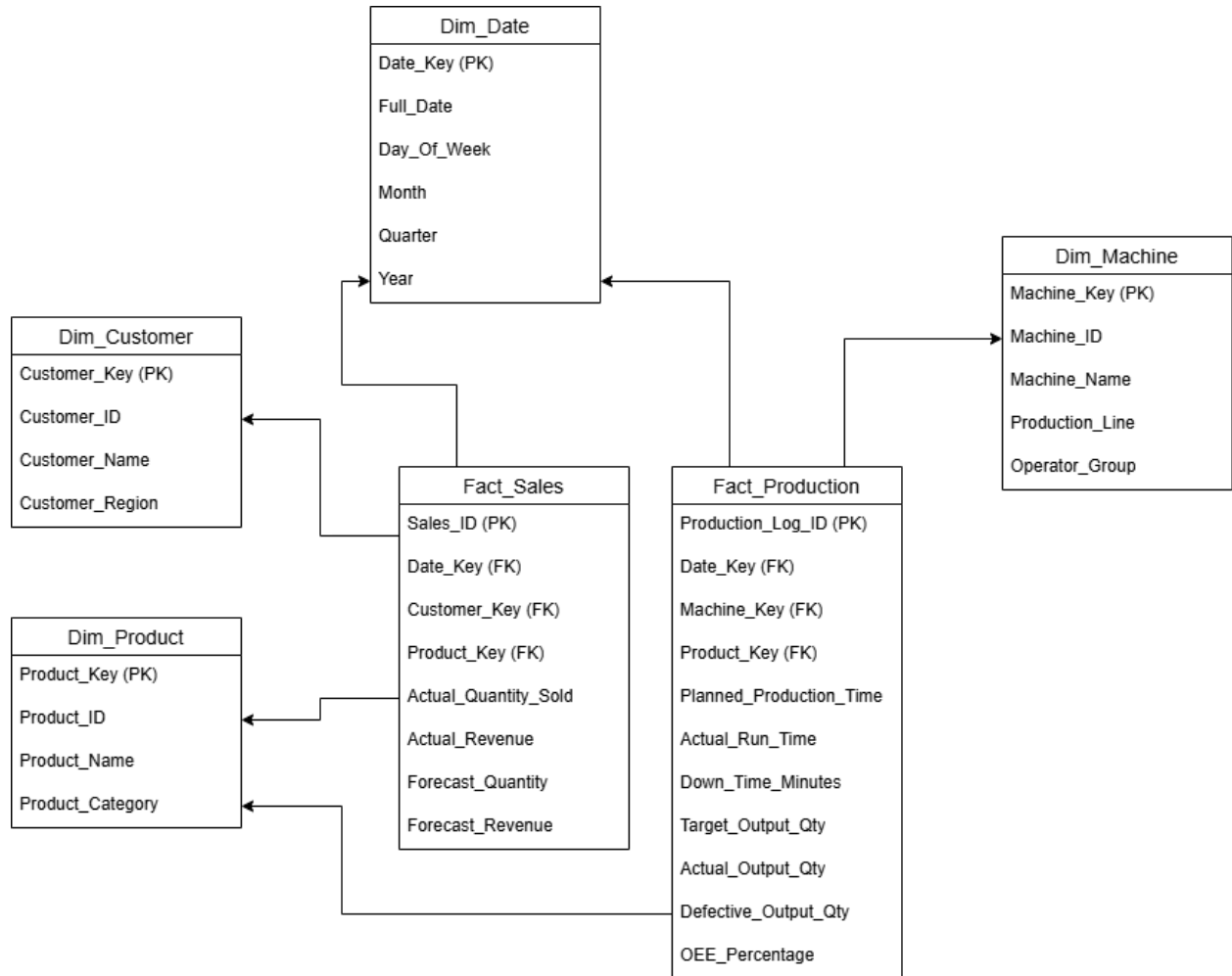


Figure 4: Galaxy Schema

tuto 4



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3843 - SPECIAL TOPIC IN DATA ENGINEERING

SECTION 2

CLAUDE/AI AGENT & PIPELINE

LECTURER: DR. ARYATI BINTI BAKRI

NAME	MATRIC NUMBER
NURUL IKA SYAFINY BINTI AZHAR	A23CS0164
DAMIYA AINA BINTI BASIR ABD SHAMMAD	A23CS0220

a. What is AI-Assisted Data Engineering

According to data infrastructure paradigms outlined by lakeFS, AI-Assisted Data Engineering represents a fundamental shift from traditional, rigid, rule-based pipeline management to an autonomous, semantic-driven data lifecycle framework. It is the practice of integrating Large Language Models (LLMs) and specialized AI Agents directly into the Extract, Transform, and Load (ETL) pipeline architecture.

Historically, data engineering relied entirely on human developers manually defining schema structures, tracking layout exceptions, writing custom RegEx filters, and mapping data formats. AI-assisted data engineering redefines this by leveraging artificial intelligence to handle the "technical drudgery", the unstructured, unpredictable string variables, mixed data styles, and unexpected log variants allowing engineering teams to scale operational infrastructure seamlessly.

b. What is Ai Agent Used in The Tutorial?

There will be two different methods included in this report since each member followed a different tutorial. Explanation will be done for both to understand what was done to produce the AI-assisted data pipelines.

1. Gemini (Google GenAI)

In this tutorial, our framework elevates Gemini (Google GenAI framework) into an active, autonomous AI Data Engineering Agent. Rather than acting as a passive assistant outside the system, Gemini was integrated directly into the automated execution runtime of the pipeline to improve the data quality of raw dataset. In this tutorial, a Netflix sample dataset with some messy data from Kaggle is used. The agent follows a dynamic loop cycle comprising four core phases:

1. **Observe (Data Profiling):** The pipeline ingests the raw, unclean dataset and automatically constructs a text-based semantic profile including schema shapes, columns, null value distribution, and a slice of rows. It passes this context directly into the agent.
2. **Plan (Quality Assessment):** Gemini reads the profile semantically. Instead of matching text line-by-line, it reasons through data quality discrepancies such as discovering that rows 6, 14, and 20 use mixed-casing like tv show and movie, or noticing that column contains unstandardized text-based and numerical date fragments.
3. **Execute (Dynamic Script Generation):** The agent dynamically engineers a targeted, custom Python Pandas cleaning script tailored specifically to the structural anomalies identified in that specific data batch.

4. **Evaluate (Validation & Load):** The pipeline programmatically executes the agent's code, validates it against a data framework sandbox, converts it to an analytical matrix, and persists it into storage.

2. ChatGPT (AI Assistant)

In this lesson, the K-pop Album Sales Data Engineering Pipeline was developed and improved with the help of ChatGPT, an AI-assisted data engineering assistant. ChatGPT functions as a collaborative assistant that assists the data engineer in designing, implementing, debugging, and documenting the pipeline process, in contrast to a completely autonomous AI agent that runs directly inside the pipeline runtime.

Throughout the data engineering lifecycle, the AI support adheres to a number of crucial tasks:

1. **Planning (Pipeline Design):** ChatGPT assists in designing the overall pipeline architecture by suggesting the implementation of the Medallion Architecture approach (Bronze → Silver → Gold). It helps define the responsibilities of each layer, data flow, folder structure, and processing sequence required for building a structured data engineering workflow.
2. **Code Generation and Development Assistance:** ChatGPT helps generate Python scripts using Pandas for each pipeline component:
 - Bronze layer for raw CSV ingestion
 - Silver layer for data cleaning and feature engineering
 - Gold layer for business analytics aggregation
 - AI Quality Check module for data validation

The generated code is then reviewed, tested, and modified by the data engineer according to the dataset requirements.

3. **Debugging and Problem Solving:** During pipeline execution, ChatGPT assists in identifying and resolving technical issues such as:
 - incorrect column references
 - missing files or paths
 - data type inconsistencies
 - execution errors
 - schema mismatches between pipeline layers

By analyzing error messages and code structure, ChatGPT provides possible solutions and improvements to maintain pipeline reliability.

4. **Data Quality Improvement :** ChatGPT supports the implementation of data validation processes by suggesting checks for:

- missing values
- duplicate records
- invalid numerical values
- inconsistent data formats
- unexpected dataset structures

This improves the reliability of the Silver and Gold layer outputs before generating business insights.

5. **Documentation and Project Enhancement:** ChatGPT assists in producing technical documentation including:

- README documentation
- pipeline explanation
- use case description
- architecture explanation
- report summaries

c. Use Case Diagram

There will be two different use cases included in this report since each member followed a different tutorial. Explanation will be done for both to understand what was done to produce the AI-assisted data pipelines.

1. Gemini (Google GenAI)

This use case diagram in Figure 1 represents an automated AI-assisted data pipeline that functions as an assembly line to clean messy data. On the left side, the Data Source (the user or engineer) kicks off the process by providing the raw Netflix dataset into the pipeline box through Data Extraction, which handles the initial file reading. Inside the pipeline box, the system coordinates three closely linked operations: extracting the data, transforming it, and cleaning it. Instead of relying on a human to write hardcoded rules, the system passes a summary of the data out to the Gemini AI Agent on the right side; Gemini analyzes the problems and sends back a custom cleaning plan during Data Transformation. This plan is then dynamically included during the Data Loading and Data Cleaning steps to automatically fix issues like mixed date formats and inconsistent text casing. Once the cleaning process is complete, the final, standardized data is pushed directly out of the pipeline box to Google Drive Storage, delivering a perfectly polished CSV file back to the user without any manual coding required.

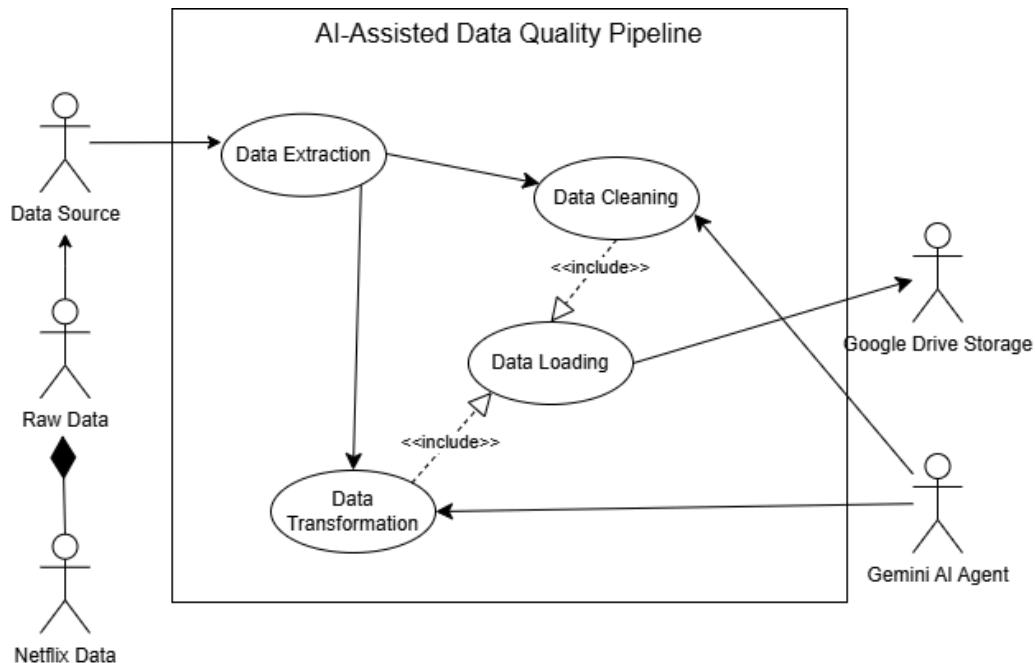


Figure 1: AI-Assisted Data Quality Pipeline Use Case Diagram

2. AI Assistant (CHATGPT)

Figure 2 below shows how the AI Assistant (ChatGPT) and the Data Engineer communicate. Uploading the raw CSV dataset, creating the Medallion Architecture workflow, carrying out the Bronze, Silver, and Gold layers, verifying the quality of the data, producing reports, and managing the entire pipeline automation are all under the purview of the Data Engineer. In the meantime, ChatGPT serves as an AI-assisted support tool that assists the data engineer with code debugging, pipeline planning, data quality checking, and documentation enhancement. The automated pipeline execution includes the main processing stages from raw data ingestion in the Bronze layer, data cleaning and feature engineering in the Silver layer, business analytics generation in the Gold layer, and final quality validation before producing analytical outputs.

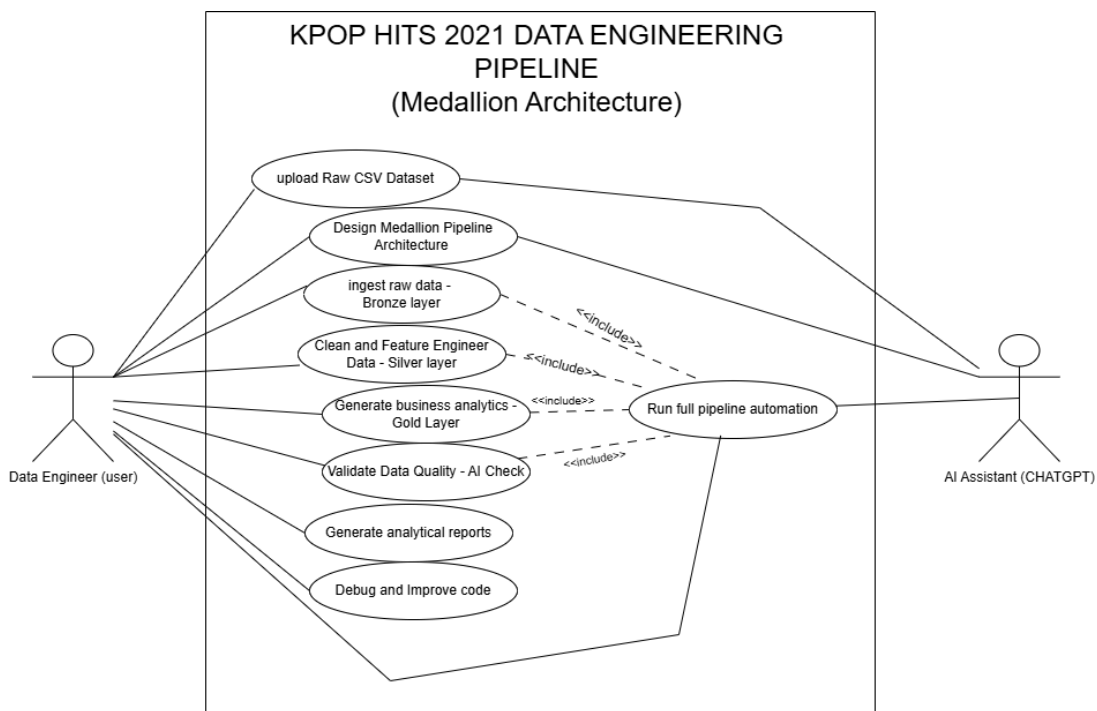


Figure 2: KPOP Hits 2021 Data Engineering Pipeline Use Case Diagram

d. Steps To Do Prompt And Coding

There will be two different coding steps included in this report since each member followed a different tutorial. Explanation will be done for both to understand what was done to produce the AI-assisted data pipelines.

1. Gemini (Google GenAI)

The development of our autonomous data pipeline was conducted systematically, dividing the engineering lifecycle into distinct operational phases within an integrated Google Colab environment. Rather than manually writing hardcoded data validation scripts, we engineered a dynamic framework that forces Gemini (via the modern Google GenAI library) to analyze an ingested profile and execute its own data remediation logic natively.

The final implemented pipeline follows a clear, production-grade Extract, Transform, and Load (ETL) sequential architecture:

Phase 1: Ingesting Unclean Data (Extract): Interactively fetches local datasets and applies fallback decoding exceptions to build a structured computational data profile. This phase acts as a defensive barrier at the entry point of your pipeline. Its primary objective is to pull an external, unstructured binary file into Python's volatile memory and extract it into a readable data frame, regardless of faulty formatting or legacy encodings. The first step is shown in Figure 2 where `files.upload()` opens an interactive browser widget allowing manual ingestion of a local storage file. Once uploaded, the raw data is captured purely as an unformatted sequence of binary bits (`file_bytes`).

```
print("--- PHASE 1: INGESTING UNCLEAR DATA ---")
# 1. Upload your local messy storage file
print("Please upload your unclean dataset (.csv or .txt):")
uploaded = files.upload()

if uploaded:
    filename = list(uploaded.keys())[0]
    file_bytes = uploaded[filename] # Get the byte content of the upload
```

Figure 2: Upload Raw File Code

Next, we apply a multi-tier exception handling hierarchy. The raw binary stream is encapsulated inside `io.BytesIO()` so that the Pandas engine can process it like an open file.

2. It first tests if the file is a compressed Microsoft Excel spreadsheet (`read_excel`).
3. If it catches an error, it treats it as a standard CSV decoded in modern UTF-8.

4. If it throws a `UnicodeDecodeError` (which happens when Excel exports special symbols using older Windows formats), it falls back to Latin-1 (cp1252) encoding. This triaging method avoids system crashes during early execution

This process is captured in Figure 3.

```
try:
    # Attempt 1: Try reading as an Excel file (common if .csv is mislab
    df_messy = pd.read_excel(io.BytesIO(file_bytes))
    print(f"Successfully read '{filename}' as an Excel file.")
except Exception as e_excel:
    print(f"Could not read '{filename}' as an Excel file. Error: {e_exc
    try:
        # Attempt 2: Try reading as CSV with utf-8 encoding
        df_messy = pd.read_csv(io.BytesIO(file_bytes), encoding='utf-8'
        print(f"Successfully read '{filename}' as a UTF-8 CSV file.")
    except UnicodeDecodeError as e_utf8:
        print(f"Could not read '{filename}' as a UTF-8 CSV file. Error:
        try:
            # Attempt 3: Try reading as CSV with latin1 encoding (commo
            df_messy = pd.read_csv(io.BytesIO(file_bytes), encoding='la
            print(f"Successfully read '{filename}' as a Latin-1 CSV fil
        except Exception as e_latin1:
            print(f"Could not read '{filename}' as a Latin-1 CSV file.
            # Fallback: If all specific attempts fail, raise a generic
            raise ValueError(f"Failed to read '{filename}'. It is neith
```

Figure 3: Multi-tier Exception Handling Code

After loading the data successfully, the pipeline summarizes the state of the dataset into a single text block called a Metadata Profile. It records column names, lengths, a text slice of the first 5 records, and a statistical aggregation of missing null values (`.isnull().sum()`) as shown in Figure 4.

```
# Print the messy data profile for the Agent to see
data_profile = f"""
Dataset Columns: {list(df_messy.columns)}
Row Count: {len(df_messy)}
Sample Data (First 5 rows):
{df_messy.to_string()}

Missing Values per column:
{df_messy.isnull().sum().to_string()}
"""

print("\n--- MESSY DATA RECEIVED ---")
print(df_messy)
```

Figure 4: Metadata Profile Code

Phase 2: AI Agent Analysis & Script Generation (Transform - Decision): Feeds the metadata profile into Gemini alongside strict prompt constraints to generate a standardized code payload. This phase represents the cognitive decision-making hub of the architecture. Instead of processing raw data matrix matrices directly, the AI Agent evaluates the high-level semantic metadata profile.

First, initialize the modern Google GenAI Client wrapper using the code shown in Figure 5. By passing the `api_key` directly into the object, the system securely grants the script programmatic authorization to connect to Google's cloud-hosted Gemini models.

```
client = genai.Client(api_key=api_key_secret)
```

Figure 5: Google GenAI Client Wrapper

Next, sets up strict operational constraints for the AI model as coded in Figure 6. Its function is to:

- `response_mime_type="application/json"`: Crucial for software engineering. It completely strips conversational text responses (like *"Sure, here is your code!"*) and forces Gemini to respond purely in structured JSON arrays that a machine can parse.
- `temperature=0.2`: A low value forces the LLM to behave deterministically, minimizing randomness or "hallucinations".

```
config=types.GenerateContentConfig(  
    response_mime_type="application/json",  
    temperature=0.2  
)
```

Figure 6: Operational Constraints for AI Model

The metadata profile is embedded directly inside an explicitly constrained engineering prompt instruction. The `gemini-2.5-flash` model reads it, diagnoses data quality issues such as unstandardized dates or lowercase strings, plans remediation steps, and outputs the result. The pipeline takes that raw JSON string and converts it back into an active Python dictionary using `json.loads()`. The process for these steps are captured in the code in Figure 7.

```
agent_prompt = f"""  
You are an expert AI Data Engineering Agent. Analyze the following profile  
Identify data quality issues (such as missing values, duplicate rows, non  
  
Write an automated, complete, and robust python code snippet using the pa  
  
CRITICAL IMPLEMENTATION RULE:  
1. Do not attempt to fix string text encoding errors using '.encode('lati  
2. For the 'date_added' column, you MUST standardize all various formats  
3. After parsing, you MUST explicitly format it to a uniform string date  
  
Data Profile:  
{data_profile}  
  
Your response must strictly be valid JSON matching this schema:  
{{  
    "analysis": "A brief explanation of the data quality issues found.",  
    "cleaning_code": "The raw Python code string to clean a dataframe named  
}}  
"""
```

```
try:  
    response = client.models.generate_content(  
        model='gemini-2.5-flash',  
        contents=agent_prompt,  
        config=types.GenerateContentConfig(  
            response_mime_type="application/json",  
            temperature=0.2  
        ),  
    )  
  
    # Parse the agent's decisions  
    agent_output = json.loads(response.text)
```

Figure 7: AI Prompt Processes

Phase 3: Pipeline Execution (Transform - Application): Copies the active memory dataframe and dynamically interprets the AI-generated instructions via a restricted execution loop (`exec()`). In this stage, the transformation strategy planned by the AI agent in Phase 2 is programmatically applied to your dataset.

The first step in this phase is an essential isolation technique in data engineering applied to the code in Figure 8. It clones the raw dataset in memory, ensuring that the original data source (`df_messy`) remains completely untouched and immutable. If an execution fails, the data is preserved safely.

```
df = df_messy.copy()
```

Figure 8: Raw Dataset Cloning

The pipeline then extracts the raw text string stored under the "cleaning_code" key from Gemini's JSON payload and feeds it to Python's internal `exec()` (Execute) function. The `exec()` statement takes a plain text string and converts it live into functional, operational Python code inside the system runtime. This allows the AI's cleaning rules (such as running multi-format date vector conversions like `format='mixed'`) to execute dynamically on the isolated `df` object. This process is implemented in the code in Figure 9.

```
exec(agent_output['cleaning_code'])
```

Figure 9: Extracting Cleaning Code

Phase 4: Loading Clean Data (Load): Generates interactive analytical displays and outputs an immutable CSV artifact (`agent_cleaned_output.csv`) to local persistent storage. The final stage closes the ETL pipeline by presenting the finalized, structured dataset and saving it to local disk storage.

The script in Figure 10 calls Colab's interactive HTML rendering system (`display(df)`) to generate an aesthetic spreadsheet visual interface matrix, allowing stakeholders to visually inspect that the date formats, missing null blocks, and text strings have been completely standardized.

```
print("\n--- PHASE 4: LOADING CLEAN DATA ---")
print("✓ Pipeline executed successfully. Final Clean Dataframe:")
display(df)
```

Figure 10: Code to Show Pipeline Executed Successfully

Finally, the uniform dataframe is serialized and converted into an immutable comma-separated values (.csv) spreadsheet schema artifact named `agent_cleaned_output.csv` and written onto the physical drive block as coded in Figure

11. Setting `index=False` eliminates systemic numerical tracking row errors during subsequent imports.

```
df.to_csv("agent_cleaned_output.csv", index=False)
print("\n✓ Saved agent-cleaned data to 'agent_cleaned_output.csv'")
```

Figure 11: Exporting Cleaned Dataset

2. ChatGPT (AI Assistant) Assisted K-pop Hits 2021 Data Pipeline Development

The development of the AI-Assisted K-pop Hits 2021 Data Pipeline was implemented using Python, Pandas, and Streamlit following the Medallion Architecture framework. Unlike traditional ETL pipelines where data moves directly from extraction to transformation, this project separates the data lifecycle into multiple layers:

- Bronze Layer: Raw Data Storage
- Silver Layer: Data Cleaning and Feature Engineering
- Gold Layer: Business Analytics
- AI Quality Check: Data Validation

ChatGPT was integrated as an AI Assistant during the development process to assist with pipeline design, debugging, code improvement, data quality validation logic, and feature engineering decisions.

Phase 1: Data Ingestion (Bronze Layer) : Ingesting the raw dataset into the Bronze Layer is the main goal of the K-pop Hits 2021 data pipeline's initial phase. The Bronze Layer serves as the first storage layer in a Medallion Architecture, where raw data is gathered and kept unaltered. This phase's goal is to produce a trustworthy replica of the original Kaggle dataset prior to any cleaning or transformation procedures.

Python Pandas was used to load the raw K-pop Hits 2021 CSV dataset for this research, which included music variables including song title, artist, danceability, energy, loudness, tempo, and duration. The Bronze Layer guarantees the preservation of the original dataset and offers a reliable source for the subsequent transformation phase.

Prompt Used:

```
I am developing an AI-Assisted K-pop Hits 2021 Data Engineering Pipeline using Medallion Architecture.

Create a Python Bronze Layer script using Pandas.

Requirements:
- Read raw CSV dataset from data/raw/KPopHits2021.csv
- Store an exact copy into data/bronze/kpop_sales_raw.csv
- Do not perform any cleaning or transformation
- Add validation to check whether file exists
- Display dataset rows, columns, column names, and sample data
- Include proper error handling
- Add comments explaining each section
```

Figure 12 : ChatGPT Prompt for Bronze Layer Development

Code Implementation

The Bronze Layer script (bronze.py) was developed to automate raw data ingestion. The script checks whether the input dataset exists, creates the Bronze directory automatically, loads the CSV file, displays dataset information, and saves the raw copy into the Bronze storage location.

The purpose of keeping the Bronze Layer unchanged is to maintain data traceability. If any issue occurs during later transformation stages, engineers can always return to the original raw dataset.

```
bronze.py X silver.py requirements.txt app.py README.md .gitignore
src > bronze.py > ...
1 # bronze.py
2 # Bronze Layer: Raw Data Ingestion (no transformation)
3
4 import os
5 import pandas as pd
6
7
8 INPUT_FILE = "data/raw/KPopHits2021.csv"
9 OUTPUT_FILE = "data/bronze/kpop_sales_raw.csv"
10
11
12 def run_bronze():
13
14     try:
15         if not os.path.exists(INPUT_FILE):
16             raise FileNotFoundError(f"Input file not found: {INPUT_FILE}")
17
18         os.makedirs(os.path.dirname(OUTPUT_FILE), exist_ok=True)
19
20         df = pd.read_csv(INPUT_FILE)
21
22         print("\n[Bronze Layer]")
23         print(f"Rows: {df.shape[0]} | Columns: {df.shape[1]}")
24
```

```

25     print("\nColumns:")
26     print(df.columns.tolist())
27
28     df.to_csv(OUTPUT_FILE, index=False)
29
30     print("\nBronze layer completed successfully.")
31     print(f"Saved: {OUTPUT_FILE}")
32
33     except Exception as e:
34         print("\nBronze Layer Error:", str(e))
35
36
37 if __name__ == "__main__":
38     run_bronze()

```

Figure 13 :Bronze Layer Code Implementation in VS Code

Bronze Layer Result

After generating the Bronze layer code using ChatGPT, the script was executed to ingest the original K-pop Hits 2021 dataset.

The Bronze layer successfully loaded the raw Kaggle dataset without applying any transformation or cleaning process. The purpose of this stage is to maintain the original dataset structure before any data processing occurs.

The execution result shows that the dataset contains:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  JUPYTER
PS C:\Users\LENOVO\Downloads\kpop-data-pipeline> python src\bronze.py

[Bronze Layer]
Rows: 50 | Columns: 14

Columns:
['Unnamed: 0', 'title', 'artist/s', 'danceability', 'energy', 'key', 'loudness', 'mode', 'speechiness', 'acousticness', 'valence', 'tempo',
'duration_ms', 'time_signature']

Bronze layer completed successfully.
Saved: data/bronze/kpop_sales_raw.csv

```

Figure 13 : Bronze Layer Execution Result

Phase 2: Data Transformation and Feature Engineering (Silver Layer) : The Bronze dataset is cleaned up and organized in the second stage. By eliminating duplicate entries, addressing missing values, standardizing text forms, correcting data kinds, and developing new analytical tools, the Silver Layer is in charge of enhancing data quality.

During this stage, synthetic characteristics were created because the original K-pop Hits 2021 dataset lacked business metrics like streaming count, sales count, and country region. The dataset can support analytical processing in the Gold Layer thanks to these extra features.

Prompt Used:

```
I have a Bronze Layer dataset from a K-pop Hits 2021 Kaggle dataset.

Create a Python Silver Layer script using Pandas.

Requirements:
- Read data/bronze/kpop_sales_raw.csv
- Remove duplicate rows
- Handle missing values
- Standardize text columns
- Fix incorrect data types
- Create synthetic analytical features:
  - streaming_count
  - sales_count
  - fan_preorder_count
  - country_region

Save the cleaned dataset into:
data/silver/kpop_sales_clean.csv

Include:
- validation checks
- comments
- error handling
```

Figure 14 : ChatGPT Prompt for Silver Layer Development

Code Implementation

The Silver Layer script (silver.py) cleans and enriches the raw K-pop dataset. The script improves dataset quality by applying transformation rules while maintaining the original data structure from Bronze Layer.

AI assistance was used during development to generate transformation logic, debug errors, and improve code structure. The AI acted as a coding assistant that supported the creation of reusable cleaning operations.

```
src > silver.py > ...
1  # silver.py
2  # Silver Layer: Cleaning + Feature Engineering
3
4  import os
5  import pandas as pd
6  import numpy as np
7
8
9  INPUT_FILE = "data/bronze/kpop_sales_raw.csv"
10 OUTPUT_FILE = "data/silver/kpop_sales_clean.csv"
11
12
13 def run_silver():
14
15     try:
16         if not os.path.exists(INPUT_FILE):
17             raise FileNotFoundError("Bronze file missing")
18
19         df = pd.read_csv(INPUT_FILE)
20
21         print("\n[Silver Layer]")
22         print(f"Original shape: {df.shape}")
23
24         # -----
25         # Clean duplicates
26         # -----
27         df = df.drop_duplicates()
28
```

```
src > silver.py > ...
13 def run_silver():
28
29     # -----
30     # Missing values
31     # -----
32     df.fillna({
33         col: "unknown" for col in df.select_dtypes(include="object").columns
34     }, inplace=True)
35
36     for col in df.select_dtypes(include=["number"]).columns:
37         df[col] = df[col].fillna(df[col].median())
38
39     # -----
40     # Standardize text
41     # -----
42     for col in df.select_dtypes(include="object").columns:
43         df[col] = df[col].str.lower().str.strip()
44
```

```

src > silver.py > ...
13 def run_silver():
45     # -----
46     # Feature Engineering
47     # -----
48     np.random.seed(42)
49     n = len(df)
50
51     df["streaming_count"] = np.random.randint(5000, 8000000, n)
52     df["sales_count"] = np.random.randint(1000, 2000000, n)
53
54     df["fan_preorder_count"] = (
55         df["sales_count"] * np.random.uniform(0.1, 0.4, n)
56     ).astype(int)
57
58     df["country_region"] = np.random.choice(
59         ["South Korea", "Japan", "USA", "Global", "SEA"], n
60     )
61
62     print("Cleaning completed")
63     print(f"Final shape: {df.shape}")
64
65     df.to_csv(OUTPUT_FILE, index=False)
66
67     print("Silver layer saved.")
68
69     except Exception as e:
70         print("Silver Layer Error:", str(e))
71
72
73 if __name__ == "__main__":
74     run_silver()

```

Figure 15 : Silver Layer Python Implementation Code

Silver Layer Result

After generating the Silver Layer code using ChatGPT, the script was executed to clean and transform the raw dataset from the Bronze Layer. This phase improved the dataset quality by applying data cleaning techniques and preparing the data for analytical processing.

The Silver Layer successfully removed duplicate records, handled missing values, standardized data formats, and generated additional analytical attributes required for business analysis. Since the original K-pop Hits 2021 dataset mainly contains music characteristics, synthetic features such as streaming count, sales count, fan preorder count, and country region were created to support the Gold Layer analysis.

The execution result shows that:

```

PS C:\Users\LENOVO\Downloads\kpop-data-pipeline> python src\silver.py

[Silver Layer]
Original shape: (50, 14)
C:\Users\LENOVO\Downloads\kpop-data-pipeline\src\silver.py:33: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to 'include' to select them, or to 'exclude' to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
  col: "unknown" for col in df.select_dtypes(include="object").columns
C:\Users\LENOVO\Downloads\kpop-data-pipeline\src\silver.py:42: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to 'include' to select them, or to 'exclude' to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
  for col in df.select_dtypes(include="object").columns:
Cleaning completed
Final shape: (50, 18)
Silver layer saved.

```

Figure 16 : Silver Layer Execution Result

Phase 3: Business Analytics Generation (Gold Layer) : The Gold Layer converts the cleaned Silver dataset into analytical datasets for reporting and decision-making. Instead of storing detailed records, this layer produces aggregated business insights.

The Gold Layer generates summaries such as:

- Artist performance
- Country analysis
- Top songs ranking
- Overall engagement metrics

These outputs are designed to support visualization through the Streamlit dashboard.

Prompt Used:

```

Create a Python Gold Layer script for my K-pop Hits 2021 data pipeline.

Input:
data/silver/kpop_sales_clean.csv

Requirements:
Generate analytical outputs:

1. Artist sales summary
2. Country sales summary
3. Top songs ranking
4. Engagement KPI summary

Save results into:
data/gold/

Include:
- Pandas aggregation
- sorting
- comments
- error handling

```

Figure 17 : ChatGPT Prompt for Gold Layer Development

Code Implementation

The Gold Layer (gold.py) applies aggregation techniques using Pandas groupby operations. The cleaned dataset is converted into business-friendly tables that can be used by dashboards and reports.

```
src > gold.py > ...
1  # gold.py
2  # Gold Layer: Business Analytics
3
4  import os
5  import pandas as pd
6
7
8  INPUT_FILE = "data/silver/kpop_sales_clean.csv"
9  OUTPUT_DIR = "data/gold"
10
11
12  def run_gold():
13
14      try:
15          if not os.path.exists(INPUT_FILE):
16              raise FileNotFoundError("Silver file missing")
17
18          os.makedirs(OUTPUT_DIR, exist_ok=True)
19
20          df = pd.read_csv(INPUT_FILE)
21
22          print("\n[Gold Layer]")
23          print(f"Rows: {df.shape[0]}")
24
25          artist_col = "artist/s"
26
```

```
src > gold.py > ...
12  def run_gold():
13
14      # -----
15      # Artist Summary
16      # -----
17      artist_summary = df.groupby(artist_col).agg(
18          total_sales=("sales_count", "sum"),
19          total_streams=("streaming_count", "sum"),
20          avg_streams=("streaming_count", "mean")
21      ).sort_values("total_sales", ascending=False)
22
23      artist_summary.to_csv(f"{OUTPUT_DIR}/artist_summary.csv")
24
25      # -----
26      # Country Summary
27      # -----
28      country_summary = df.groupby("country_region").agg(
29          total_sales=("sales_count", "sum"),
30          total_streams=("streaming_count", "sum")
31      ).sort_values("total_sales", ascending=False)
32
33      country_summary.to_csv(f"{OUTPUT_DIR}/country_summary.csv")
34
```

```

48 # -----
49 # Top Songs
50 # -----
51 top_songs = df.groupby("title").agg(
52     total_sales=("sales_count", "sum")
53 ).sort_values("total_sales", ascending=False)
54
55 top_songs.to_csv(f"{OUTPUT_DIR}/top_songs.csv")
56
57 print("Gold layer completed.")
58
59 except Exception as e:
60     print("Gold Layer Error:", str(e))
61
62
63 if __name__ == "__main__":
64     run_gold()

```

Figure 18 : Gold Layer Python Implementation Code

Gold Layer Result

After generating the Gold Layer code using ChatGPT, the script was executed to transform the cleaned Silver dataset into analytical outputs. The purpose of this stage is to convert processed data into meaningful business insights that can support reporting and visualization.

The Gold Layer applies aggregation techniques to summarize K-pop Hits 2021 data based on different analytical perspectives. Instead of storing individual song records, this layer creates business-ready datasets such as artist performance analysis, country distribution, top songs ranking, and overall engagement metrics.

The execution result confirms that the Gold Layer successfully generated:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  JUPYTER

PS C:\Users\LENOVO\Downloads\kpop-data-pipeline> python src\gold.py

[Gold Layer]
Rows: 50
Gold layer completed.

```

Figure 19 : Gold Layer Execution Result

Phase 4: AI Data Quality Validation : The final phase introduces AI-assisted quality checking. The purpose of this stage is to validate whether the processed Silver dataset meets expected data quality standards.

The AI Quality Check script automatically evaluates:

- missing values
- duplicate records
- empty columns
- incorrect data types
- invalid numerical values

AI improves this process by assisting engineers in identifying potential issues and suggesting improvements instead of manually checking every column.

Prompt Used:

```
Create an AI-assisted data quality checking Python script.

Dataset:
data/silver/kpop_sales_clean.csv

Check:

- missing values
- duplicate rows
- empty columns
- data types
- invalid numeric values

Generate:
- console quality report
- text report output

Add comments explaining the validation process.
```

Figure 20 : ChatGPT Prompt for Quality Check Development

Code Implementation

The AI Quality Check module validates the final cleaned dataset before analytics usage. This improves reliability because incorrect or incomplete data can be detected before reaching reporting systems.

```

src > ai_quality_check.py > ...
15 def run_quality_check():
101
102     with open(REPORT_FILE, "w", encoding="utf-8") as f:
103         f.write("\n".join(report_lines))
104
105         print("\n=== QUALITY CHECK COMPLETED ===")
106         print(f"Report saved to: {REPORT_FILE}")
107
108     except FileNotFoundError as fnf:
109         print("\n❌ File Error:", fnf)
110
111     except Exception as e:
112         print("\n❌ Unexpected Error:", str(e))
113
114
115 # -----
116 # Run check
117 # -----
118 if __name__ == "__main__":
119     run_quality_check()

```

Figure 21 : AI Quality Check Python Implementation Code

AI Quality Check Result

After generating the AI Quality Check script using ChatGPT, the validation process was executed on the Silver Layer dataset. This phase ensures that the cleaned dataset meets the required data quality standards before being used for analytics and reporting.

The AI-assisted quality checking process automatically examined different quality dimensions including missing values, duplicate records, empty columns, data types, and invalid numerical values. This reduces manual inspection effort and improves reliability during pipeline execution.

The execution result shows that:

```
PS C:\Users\LENOVO\Downloads\kpop-data-pipeline> python src\ai_quality_check.py

=== AI DATA QUALITY CHECK STARTED ===
Rows: 50
Columns: 18

Missing Values:
  Unnamed: 0      0
title            0
artist/s         0
danceability      0
energy           0
key              0
loudness         0
mode             0
speechiness      0
acousticness     0
valence          0
tempo            0
duration_ms      0
time_signature   0
streaming_count  0
sales_count      0
fan_preorder_count 0
country_region   0
dtype: int64

Duplicate Rows: 0

Empty Columns: []

Empty Columns: []

Data Types:
  Unnamed: 0      int64
title           str
artist/s        str
danceability    float64
energy          float64
key             int64
loudness        float64
mode           int64
speechiness     float64
acousticness    float64
valence         float64
tempo           float64
duration_ms     int64
time_signature  int64
streaming_count int64
sales_count     int64
fan_preorder_count int64
country_region  str
dtype: object

Invalid Numeric Values (Negative Check):
{'Unnamed: 0': 0, 'danceability': 0, 'energy': 0, 'key': 0, 'loudness': 50, 'mode': 0, 'speechiness': 0, 'acousticness': 0, 'valence': 0, 'tempo': 0, 'duration_ms': 0, 'time_signature': 0, 'streaming_count': 0, 'sales_count': 0, 'fan_preorder_count': 0}

=== QUALITY CHECK COMPLETED ===
Report saved to: data/gold/data quality report.txt
```

Figure 22 : AI Quality Check Execution Result

Data_quality_report.txt

```
nts.txt  app.py  README.md  data > gold > data_quality_report.txt
1  === MISSING VALUES ===
2  Unnamed: 0 0
3  title 0
4  artist/s 0
5  danceability 0
6  energy 0
7  key 0
8  loudness 0
9  mode 0
10 speechiness 0
11 acousticness 0
12 valence 0
13 tempo 0
14 duration_ms 0
15 time_signature 0
16 streaming_count 0
17 sales_count 0
18 fan_preorder_count 0
19 country_region 0
20 dtype: int64
21
22 === DUPLICATES ===
23 Total duplicate rows: 0
24
```

```
22 === DUPLICATES ===
23 Total duplicate rows: 0
24
25 === EMPTY COLUMNS ===
26 []
27
```

```

28  === DATA TYPES ===
29  Unnamed: 0          int64
30  title               str
31  artist/s           str
32  danceability        float64
33  energy              float64
34  key                 int64
35  loudness            float64
36  mode                int64
37  speechiness         float64
38  acousticness        float64
39  valence             float64
40  tempo               float64
41  duration_ms         int64
42  time_signature       int64
43  streaming_count      int64
44  sales_count          int64
45  fan_preorder_count  int64
46  country_region      str
47  dtype: object

48
49  === INVALID NUMERIC VALUES (NEGATIVE CHECK) ===
50  {'Unnamed: 0': 0, 'danceability': 0, 'energy': 0, 'key': 0, 'loudness': 50, 'mode': 0,
51   'speechiness': 0, 'acousticness': 0, 'valence': 0, 'tempo': 0, 'duration_ms': 0,
52   'time_signature': 0, 'streaming_count': 0, 'sales_count': 0, 'fan_preorder_count': 0}
53
54  === SUMMARY ===
55  Total rows: 50
56  Total columns: 18
57  Duplicate rows: 0

```

Figure 23 : AI Quality Check saved in Report

Phase 5: Dashboard Visualization (Streamlit Application) : After completing the Bronze, Silver, Gold, and AI Quality Check phases, a Streamlit dashboard was developed to visualize the processed K-pop Hits 2021 dataset. The purpose of this dashboard is to provide an interactive interface where users can explore the pipeline outputs without directly opening CSV files.

The application reads the analytical datasets generated by the Gold Layer and presents important insights through KPI cards, tables, and charts. This allows stakeholders to understand song performance, artist trends, country distribution, and engagement metrics generated from the data pipeline.

The Streamlit application acts as the presentation layer of the architecture, connecting the data engineering pipeline with user-friendly analytics visualization.

Prompt Used:

```
I have created an AI-Assisted K-pop Hits 2021 Data Engineering Pipeline using Medallion Architecture.

Create a Streamlit dashboard application using Python.

Requirements:

- Load datasets from:
data/raw/KPopHits2021.csv
data/silver/kpop_sales_clean.csv
data/gold/

Create dashboard sections:
↓

1. Overview KPI Dashboard
2. Raw Data Preview
3. Silver Clean Dataset Preview
4. Gold Analytics Visualization

Display:
- total sales
- total streaming
- fan preorder count
- top artists
- country analysis
- top songs ranking

Use:
- pandas
- streamlit
↓

Include:
- error handling
- comments
- clean structure
```

Figure 24 : ChatGPT Prompt for Visualization Development

Code Implementation

The Streamlit application (app.py) was created to provide a visual representation of the completed data pipeline. The application loads the datasets generated from different layers and organizes the information into several sections.

The dashboard contains four main sections:

1. **Overview Dashboard**

Displays key performance indicators such as total sales, total streams, and preorder metrics generated from the Gold Layer.

2. **Raw Data Preview**

Allows users to view the original Kaggle dataset before any processing occurs in the Bronze Layer.

3. **Silver Data Preview**

Displays the cleaned and enriched dataset after transformation, including generated analytical attributes.

4. **Gold Analytics**

Presents business insights using charts and tables, including top artists, country analysis, and song performance ranking.

AI assistance was used during development to generate the initial Streamlit structure, debug data loading errors, and improve the dashboard organization.

```

app.py > ...
1  import streamlit as st
2  import pandas as pd
3  from pathlib import Path
4
5  # =====
6  # PAGE CONFIG
7  # =====
8
9  st.set_page_config(
10     page_title="K-pop Hits 2021 Dashboard",
11     page_icon="🎵",
12     layout="wide"
13 )
14
15 # =====
16 # TITLE
17 # =====
18
19 st.title("🎵 K-pop Hits 2021 Dashboard")
20
21 st.markdown("""
22 AI-Assisted Pipeline using Medallion Architecture:
23 Bronze → Silver → Gold
24 """)
25

```

```

26 # =====
27 # FILE PATHS
28 # =====
29
30 RAW_FILE = Path("data/raw/KPopHits2021.csv")
31 SILVER_FILE = Path("data/silver/kpop_sales_clean.csv")
32
33 ARTIST_FILE = Path("data/gold/artist_summary.csv")
34 COUNTRY_FILE = Path("data/gold/country_summary.csv")
35 TOP_SONGS_FILE = Path("data/gold/top_songs.csv")
36 ENGAGEMENT_FILE = Path("data/gold/engagement_summary.csv")
37
38 # =====
39 # LOAD DATA SAFELY
40 # =====
41
42 @st.cache_data
43 def load_data():
44     raw_df = pd.read_csv(RAW_FILE)
45     silver_df = pd.read_csv(SILVER_FILE)
46
47     artist_df = pd.read_csv(ARTIST_FILE)
48     country_df = pd.read_csv(COUNTRY_FILE)
49     top_songs_df = pd.read_csv(TOP_SONGS_FILE)
50     engagement_df = pd.read_csv(ENGAGEMENT_FILE)
51
52     return raw_df, silver_df, artist_df, country_df, top_songs_df, engagement_df
53

```

```

55 try:
56     raw_df, silver_df, artist_df, country_df, top_songs_df, engagement_df = load_data()
57
58 except Exception as e:
59     st.error(f"Data loading error: {e}")
60     st.stop()
61
62 # =====
63 # SIDEBAR
64 # =====
65
66 section = st.sidebar.radio(
67     "Navigation",
68     ["Overview", "Raw Data", "Silver Data", "Gold Analytics"]
69 )
70
71 # =====
72 # OVERVIEW (FIXED KPI LOGIC)
73 # =====
74
75 if section == "Overview":
76
77     st.header("📊 KPI Dashboard")
78
79     # FIX: engagement is metric-value format
80     engagement_map = dict(zip(
81         engagement_df["metric"],
82         engagement_df["value"]
83     ))
84

```

```

84
85     total_sales = int(engagement_map.get("total_sales", 0))
86     total_streams = int(engagement_map.get("total_streams", 0))
87     total_preorders = int(engagement_map.get("total_preorders", 0))
88
89     col1, col2, col3 = st.columns(3)
90
91     col1.metric("Total Sales", f"{total_sales:,}")
92     col2.metric("Total Streams", f"{total_streams:,}")
93     col3.metric("Total Preorders", f"{total_preorders:,}")
94
95     st.markdown("---")
96
97     st.write("Pipeline: Bronze → Silver → Gold → AI Quality Check")
98
99 # =====
100 # RAW DATA
101 # =====
102
103 elif section == "Raw Data":
104
105     st.header("📄 Raw Dataset")
106
107     st.write(f"Rows: {raw_df.shape[0]} | Columns: {raw_df.shape[1]}")
108     st.dataframe(raw_df)
109

```

```

110 # =====
111 # SILVER DATA
112 # =====
113
114 elif section == "Silver Data":
115
116     st.header("🧹 Cleaned Dataset")
117
118     st.write(f"Rows: {silver_df.shape[0]} | Columns: {silver_df.shape[1]}")
119     st.dataframe(silver_df)
120
121 # =====
122 # GOLD ANALYTICS
123 # =====
124
125 elif section == "Gold Analytics":
126
127     st.header("💡 Business Insights")

```

```

129 # -----
130 # Top Artists
131 # -----
132
133 st.subheader("Top Artists by Sales")
134
135 artist_chart = artist_df.sort_values("total_sales", ascending=False).head(10)
136
137 # FIX: artist column name safe handling
138 artist_col = "artist/s" if "artist/s" in artist_df.columns else artist_df.columns[0]
139
140 st.bar_chart(
141     artist_chart.set_index(artist_col)["total_sales"]
142 )
143
144 st.dataframe(artist_chart)
145
146 st.markdown("---")

```

```

148 # -----
149 # Countries
150 # -----
151
152 st.subheader("Top Countries by Sales")
153
154 country_chart = country_df.sort_values("total_sales", ascending=False).head(10)
155
156 st.bar_chart(
157     country_chart.set_index("country_region")["total_sales"]
158 )
159
160 st.dataframe(country_chart)
161
162 st.markdown("---")
163
164 # -----
165 # Top Songs
166 # -----
167
168 st.subheader("Top Songs by Sales")
169
170 songs_chart = top_songs_df.sort_values("total_sales", ascending=False).head(10)
171
172 st.bar_chart(
173     songs_chart.set_index("title")["total_sales"]
174 )
175
176 st.dataframe(songs_chart)

```

```

178 # =====
179 # FOOTER
180 # =====
181
182 st.markdown("---")
183
184 st.caption(
185     "K-pop Data Engineering Pipeline | Built with Python, Pandas & Streamlit"
186 )

```

Figure 25 : Streamlit Dashboard Python Implementation Code

Dashboard Execution Result

After completing the Streamlit application, the dashboard was executed using the Streamlit command. The application successfully loaded the processed datasets from the pipeline and displayed the analytical results through an interactive dashboard interface.

The dashboard demonstrates that the AI-assisted data engineering pipeline successfully transforms raw K-pop Hits 2021 data into meaningful visual insights through the complete workflow:

Bronze → Silver → Gold → Dashboard

The displayed results include:

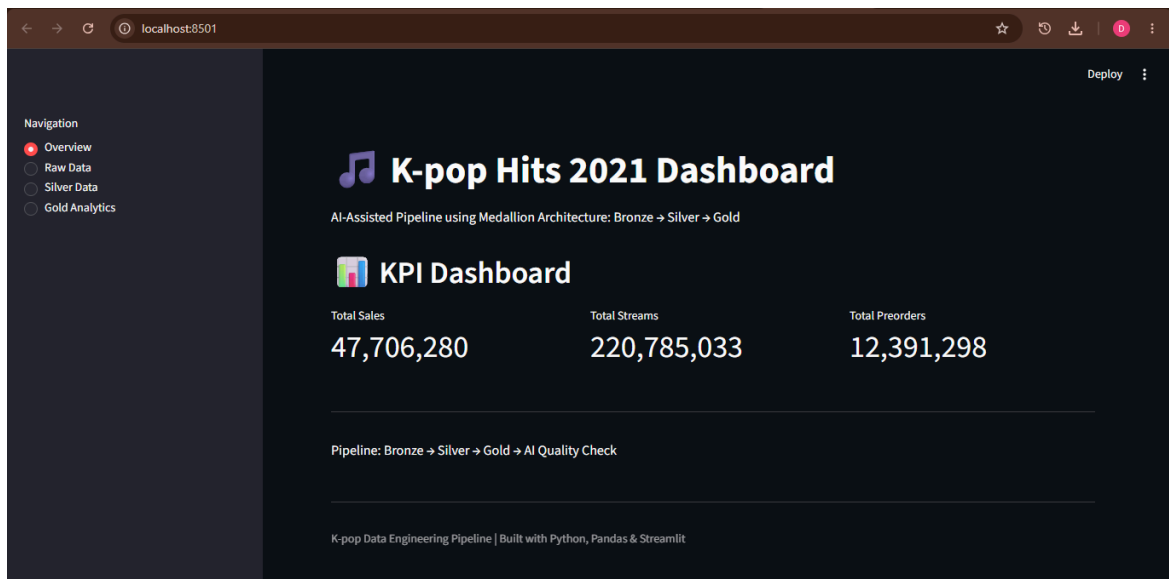


Figure 26 : Dashboard Overview

Navigation

- Overview
- Raw Data**
- Silver Data
- Gold Analytics

Raw Dataset

Rows: 50 | Columns: 14

	Unnamed: 0	title	artist/s	danceability	energy	key	loudness	mode	speechiness	acousticness	valence
0	0	Butter	BTS	0.759	0.459	8	-5.187	1	0.0948	0.0032	0.695
1	1	The Feels	TWICE	0.808	0.903	9	-2.179	1	0.0729	0.0899	0.922
2	2	Savage	aespa	0.727	0.879	9	-1.167	1	0.129	0.124	0.671
3	3	Permission to Dance	BTS	0.702	0.741	9	-5.33	1	0.0427	0.0054	0.646
4	4	In the morning	ITZY	0.846	0.814	6	-4.685	0	0.149	0.0823	0.722
5	5	Alcohol-Free	TWICE	0.688	0.909	8	-1.992	1	0.0624	0.0474	0.702
6	6	MONEY	LISA	0.826	0.553	1	-10.121	0	0.234	0.163	0.4
7	7	ELEVEN	IVE	0.828	0.731	9	-3.033	1	0.111	0.0617	0.587
8	8	Queendom	Red Velvet	0.695	0.896	5	-1.24	1	0.0847	0.179	0.508
9	9	Celebrity	IU	0.73	0.784	9	-2.62	1	0.0691	0.191	0.673

Figure 27 : Raw Data (Bronze Layer) Dashboard

Navigation

- Overview
- Raw Data
- Silver Data**
- Gold Analytics

Cleaned Dataset

Rows: 50 | Columns: 18

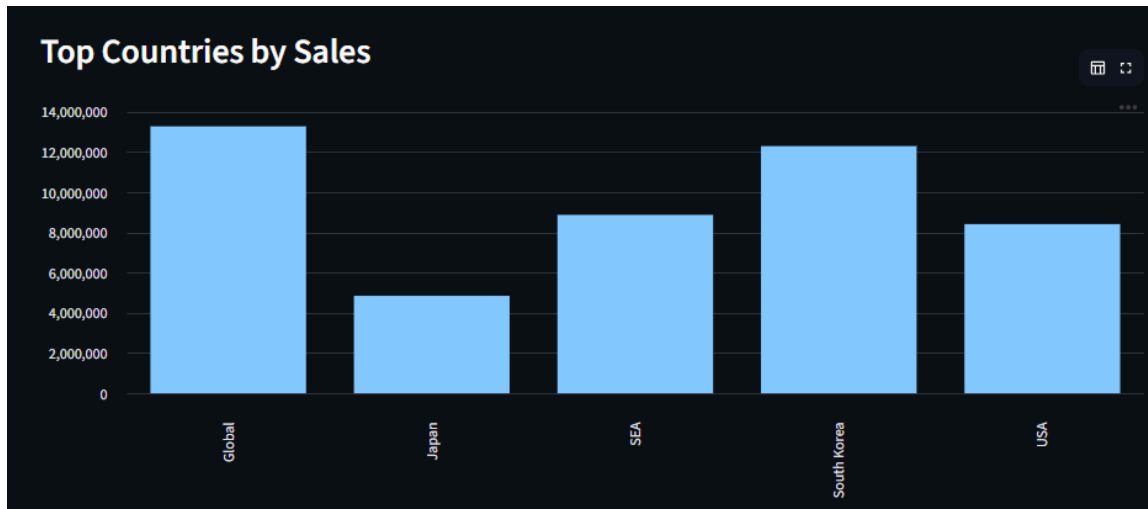
	Unnamed: 0	title	artist/s	danceability	energy	key	loudness	mode	speechiness	acousticness	valence
0	0	butter	bts	0.759	0.459	8	-5.187	1	0.0948	0.0032	0.695
1	1	the feels	twice	0.808	0.903	9	-2.179	1	0.0729	0.0899	0.922
2	2	savage	aespa	0.727	0.879	9	-1.167	1	0.129	0.124	0.671
3	3	permission to dance	bts	0.702	0.741	9	-5.33	1	0.0427	0.0054	0.646
4	4	in the morning	itzy	0.846	0.814	6	-4.685	0	0.149	0.0823	0.722
5	5	alcohol-free	twice	0.688	0.909	8	-1.992	1	0.0624	0.0474	0.702
6	6	money	lisa	0.826	0.553	1	-10.121	0	0.234	0.163	0.4
7	7	eleven	ive	0.828	0.731	9	-3.033	1	0.111	0.0617	0.587
8	8	queendom	red velvet	0.695	0.896	5	-1.24	1	0.0847	0.179	0.508
9	9	celebrity	iu	0.73	0.784	9	-2.62	1	0.0691	0.191	0.673

Figure 28 : Cleaned Dataset (Silver Layer) Dashboard



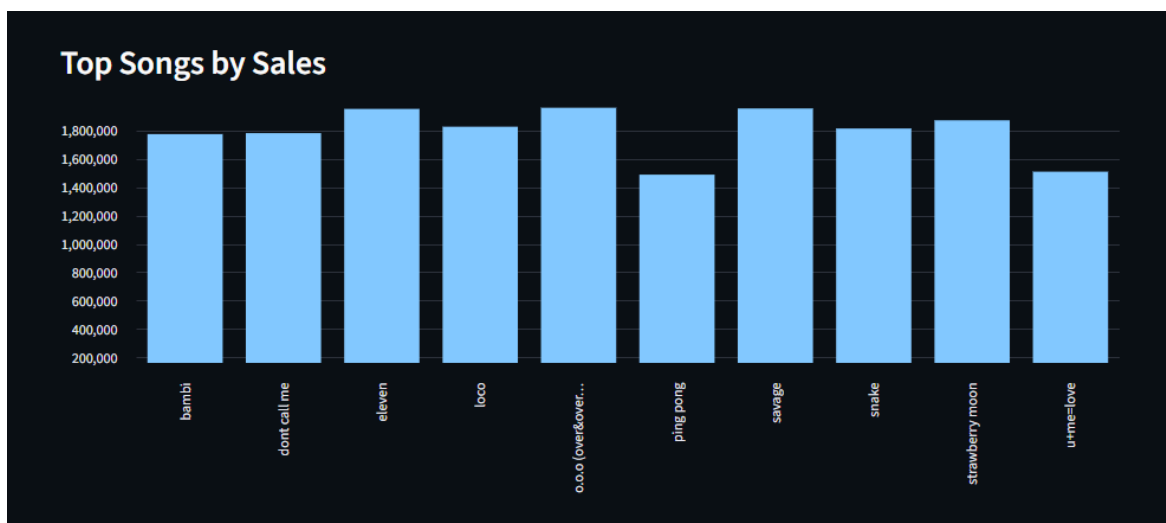
	artist/s	total_sales	total_streams	avg_streams
0	twice	3799489	14235113	4745037.6667
1	iu	2771058	10597183	3532394.3333
2	tomorrow x together	2633051	7197446	2399148.6667
3	itzy	2146695	11082007	5541003.5
4	girls planet 999	1960366	7580828	7580828
5	nct dream	1957220	13729997	4576665.6667
6	aespa	1955354	6428388	6428388
7	ive	1952224	4931484	4931484
8	medusa	1814045	7494535	7494535
9	shinee	1781488	4986505	4986505

Figure 29 : Business Analytics (Gold Layer) - Top Artist by Sales



	country_region	total_sales	total_streams
0	Global	13275270	74376781
1	South Korea	12294234	53845616
2	SEA	8875264	32629926
3	USA	8413795	42586039
4	Japan	4847717	17346671

Figure 30 : Business Analytics (Gold Layer) - Top Countries by Sales



	title	total_sales
0	o.o.o (over&over&over)	1960366
1	savage	1955354
2	eleven	1952224
3	strawberry moon	1871928
4	loco	1826665
5	snake	1814045
6	dont call me	1781488
7	bambi	1774415
8	u+me=love	1509349
9	ping pong	1488550

Figure 31 : Business Analytics (Gold Layer) - Top Songs by Sales

e. Reflection

Nurul Ika Syafiny Binti Azhar

This tutorial successfully demonstrates how shifting from rigid, rule-based coding to an autonomous AI agent workflow can revolutionize traditional data engineering tasks. By integrating the Gemini API directly into our Python pipeline, we moved past using AI as a basic code copy-paste tool and instead built an adaptive system that can independently audit a dataset's metadata, diagnose structural flaws, and execute its own custom remediation code at runtime. Overcoming real-world pipeline failures such as handling complex international text encoding bugs and parsing messy, multi-format Netflix date strings highlighted the vital importance of prompt engineering and structured JSON guardrails to ensure system predictability. Ultimately, this agentic approach delivers immense scalability for modern enterprise data systems, showing that while AI introduces non-deterministic variations, it can be safely constrained by software engineering guardrails to clean messy data autonomously without human intervention.

Damiya Aina binti Basir Abd Shammad

The development of the AI-assisted K-pop Hits 2021 data pipeline shows how integrating artificial intelligence tools like ChatGPT can greatly improve contemporary data engineering techniques. All phases of the Medallion Architecture, including Bronze, Silver, Gold, and the AI Quality Check layer, employed AI not only as a coding help but also as a design and debugging collaborator. The way raw data moves through a structured pipeline beginning with ingestion in the Bronze Layer, cleaning and feature engineering in the Silver Layer, and ultimately turning into analytical insights in the Gold Layer became more apparent to the developer. By helping with error correction, enhancing code structure, and producing artificial business features when the dataset lacked real-world metrics like sales and streaming counts, the application of AI decreased development complexity.

In addition, the integration of AI improved problem-solving efficiency during debugging stages. Instead of manually troubleshooting errors, the developer was able to use AI suggestions to quickly identify issues related to column mismatches, missing files, and incorrect aggregations. This allowed faster iteration and improved overall pipeline stability. The Streamlit dashboard further strengthened the understanding of end-to-end data engineering by visualizing the final outputs in an interactive format. This helped connect technical processing with real-world business interpretation, such as identifying top-performing artists and countries in the K-pop industry. To summarize, the research shows that AI-assisted data engineering is an augmentation tool that increases efficiency, accuracy, and scalability in contemporary data processes rather than a substitute for traditional engineering abilities.

References

Bansal, S. (2021). *Netflix Movies and TV Shows*. Wwww.kaggle.com.

<https://www.kaggle.com/datasets/shivamb/netflix-shows>

Idan Novogroder. (2024, December 16). *AI in Data Engineering: Challenges, Best Practices & Tools*. Git for Data - LakeFS. <https://lakefs.io/blog/ai-data-engineering/>

Berjamin, S. A. (n.d.). *K-Pop Hits Through the Years*. [Wwww.kaggle.com](https://www.kaggle.com).

<https://www.kaggle.com/datasets/sberj127/kpop-hits-through-the-years?resource=download&select=KPopHits2021.csv>